

Dynamic Population Size Configuration in CMA-ES via Deep Reinforcement Learning

Prakhar Gupta
Student ID: 250031444
University of St Andrews

11 August 2026

This dissertation is submitted in partial fulfilment of the requirements of the University of St Andrews for the degree of MSc Artificial Intelligence.

Abstract

The population size λ is one of the most consequential hyperparameters of the Covariance Matrix Adaptation Evolution Strategy (CMA-ES), governing the trade-off between exploration and exploitation during black-box optimisation. Existing solutions for population size control were surveyed, including restart schemes such as IPOP-CMA-ES and BIPOP-CMA-ES, and continuous heuristics such as PSA-CMA-ES and CMAES-APOP. However, they adapt λ using hand crafted rules. Designing such rules requires considerable expert knowledge, including knowing which signals are informative and how to map them to a population size decision. This dissertation investigates whether the design process can be automated, learning a population size control policy using deep reinforcement learning. A custom Gymnasium environment wraps the `modcma` implementation of CMA-ES and exposes population size as a continuous action. A PPO agent, adapted from the CleanRL reference implementation, is trained on the BBOB noiseless testbed across four candidate reward functions. The best policy is compared against a static default configuration and IPOP-CMA-ES on all 24 BBOB functions across six dimensions, using success rate, area over the convergence curve (AOCC), and empirical cumulative distribution functions (ECDF). The learned controller roughly doubles the success rate of the static default and narrows the gap to IPOP, while using substantially fewer evaluations. The results suggest that reward shaping based on normalised fitness improvement is a promising direction for DRL based population control, but the learned policy, while competitive, is not as strong as IPOP on this benchmark.

Declaration

I hereby certify that this dissertation, which is approximately 14600 words in length, has been composed by me, that it is the record of work carried out by me and that it has not been submitted in any previous application for a degree. This project was conducted by me at the University of St Andrews from June 2026 to August 2026 towards fulfilment of the requirements of the University of St Andrews for the degree of MSc Artificial Intelligence under the supervision of Nguyen Dang.

In submitting this project report to the University of St Andrews, I give permission for it to be published online. I retain the copyright in this work.

11 August 2026

Prakhar Gupta

Contents

1	Introduction	6
2	Background	7
2.1	Reinforcement Learning	7
2.1.1	The Markov Decision Process	7
2.1.2	Deep Reinforcement Learning	8
2.1.3	Proximal Policy Optimisation	9
2.2	Dynamic Algorithm Configuration	10
2.3	Evolutionary Algorithms	11
2.3.1	Genetic Algorithms	11
2.3.2	Evolution Strategies	11
2.3.3	CMA-ES	11
2.4	Connecting the Parts	13
3	Related Work	13
3.1	IPOP-CMA-ES: Increasing Population [3]	13
3.2	AP-CMA-ES: Adaptive Population [2]	14
3.3	PSA-CMA-ES: Population Size Adaptation [24]	14
3.4	CMAES-APOP: Adapting Population for Multimodal Functions [21]	15
3.5	Population Control Beyond Single-Objective CMA-ES	15
3.6	Research Gap	15
4	Methodology	16
4.1	Problem Formulation	16
4.2	State Representation	17
4.3	Reward Functions	18
4.4	Training Setup	19
4.4.1	Problem Sampling	19
4.4.2	Agent	20
4.4.3	Training Script	20
4.5	Evaluation Setup	21
4.6	Software and Libraries	21
4.6.1	CMA-ES Implementation	21
4.6.2	RL Environment	22
4.6.3	Function Benchmarks	22
4.6.4	Code Structure	22
5	Experiments and Evaluation	23
5.1	Research Questions	23
5.2	Metrics	24
5.3	Training Experiments	24
5.3.1	Phase 1	25
5.3.2	Phase 2	25
5.3.3	Phase 3	26
5.3.4	Phase 4	26
5.4	Algorithm Comparison	27
5.4.1	Success Rates	28
5.4.2	AOCC	29
5.4.3	ECDF	29

5.4.4	Generalisation Across Dimensions	30
5.5	Critical Appraisal	30
6	Future Work	31
7	Conclusion	32
A	User Manual	36
A.1	Environment Setup	36
A.2	Training a Policy	36
A.3	Evaluating a Trained Policy	37
A.4	Viewing Results	38
A.5	Reproducing the Reported Results End to End	39
B	Supplementary Training and Evaluation Data	39

1 Introduction

Black-box optimisation over non-linear, non-convex, and otherwise rugged search landscapes is a fundamental challenge in many applied fields, including hyperparameter tuning in machine learning and policy search in robotics. In black-box optimisation the objective function is unknown or expensive to evaluate analytically - the optimiser can only query it as a function that accepts a candidate solution and returns a fitness value, with no access to its underlying analytical form, gradient, or shape.

The Covariance Matrix Adaptation Evolution Strategy (CMA-ES) offers strong performance on this class of problems, particularly on ill-conditioned and non-separable functions [10]. CMA-ES operates by sampling candidate solutions from a multivariate normal distribution and iteratively refining the distribution’s mean and covariance matrix based on the fitness of the sampled population.

A central hyperparameter of CMA-ES is its population size (λ), the number of candidate solutions sampled per generation. It governs a fundamental trade-off between smaller populations, which converge quickly by exploiting the local neighbourhood of the current solutions, and larger populations, which explore more broadly and reduce the risk of premature convergence [10]. The population size is conventionally set using a fixed heuristic based on problem dimensionality. A growing body of work suggests that dynamically adapting hyperparameters during runtime, rather than fixing them in advance, can substantially improve convergence behaviour [1].

Existing approaches to dynamic population control in CMA-ES include restart based strategies such as IPOP-CMA-ES, which doubles λ on every restart [3], and BIPOP-CMA-ES, which alternates between large and small population regimes under a budget allocation scheme [9]. Continuous, restart-free adaptation strategies have also been proposed, including AP-CMA-ES, which treats λ as a self adaptive strategy parameter that responds to the local fitness landscape [2], and CMAES-APOP, which adjusts λ every S iterations based on improvement relative to the population median [22]. These approaches consistently improve on static configurations, but they remain heuristic in nature - each encoding a designer’s prior beliefs about which signals matter and how they should map onto a population size decision. None of them can improve that mapping from experience.

This dissertation investigates a fundamentally different approach: controlling the CMA-ES population size using deep reinforcement learning (DRL). The dynamic configuration of λ exhibits precisely the properties that make reinforcement learning (RL) a good fit for a control problem: the process is stateful (summarised by the current population size, step size, and recent fitness history), the payoff is delayed (overall convergence speed, not any single generation’s improvement), and the environment is cheap to simulate [16]. Because the resulting state space has continuous components, deep learning is used to approximate the policy and value functions, following the approach first demonstrated for high dimensional control problems in [19].

Recent work supports the feasibility of this approach.[31] used Proximal Policy Optimisation (PPO) to accelerate the step size adaptation of an evolution strategy and reported faster convergence than CMA-ES, particularly in high dimensional problems. [28] independently showed that an RL trained controller for CMA-ES step size adaptation, trained once across a distribution of problem instances, can outperform the algorithm’s own fixed adaptation rule at evaluation time. The broader motivation for treating this as a learning problem is rooted in dynamic algorithm configuration (DAC), where learned configuration policies have been shown to consistently outperform static, heuristic-based alternatives across a range of algorithms [1].

This dissertation implements a DRL based population size controller for CMA-ES and

evaluates the learned policy against a static default configuration and IPOP-CMA-ES [3] on the COCO [12] BBOB noiseless function suite [13]. IPOP-CMA-ES is a seminal work on population size adaptation, and remains a reference point against which the later strategies evaluate their own performance, making it an obvious choice for a baseline. While stronger, more state aware heuristics (BIPOP-CMA-ES, PSA-CMA-ES, AP-CMA-ES, and CMAES-APOP) are reviewed in Section 3, they are outside the scope of this dissertation’s evaluation. The common controller interface is built in a manner that allows them to be added later, and this is identified as future work in Section 6.

Section 2 covers the background on deep reinforcement learning, dynamic algorithm configuration, and CMA-ES. Section 3 surveys related works on population size control, and identifies the gap this dissertation addresses. Section 4 describes the implementation, Section 5 presents the experiments and evaluation, and Sections 6 and 7 discuss future work and conclusions.

2 Background

2.1 Reinforcement Learning

Reinforcement learning (RL) is a computational framework in which an agent learns to make decisions by interacting with an environment. The agent receives feedback in the form of rewards based on the actions it takes, and the goal is to learn a policy that accumulates the maximum amount of reward over time. Unlike supervised learning, where a model is trained on labelled input data, RL relies on the agent discovering good actions through trial and error. RL is thus well suited to sequential decision making problems, where the consequences of an action unfold over time and no single supervising signal is available at each step [16].

2.1.1 The Markov Decision Process

RL problems are formally modelled as a Markov Decision Process (MDP), defined as the tuple $\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$ [16]:

- $\mathcal{S}$ is the **state space**, the set of all situations the environment can be in. A single **state** $s_t \in \mathcal{S}$ is the description of the environment that the agent observes at time step t .
- $\mathcal{A}$ is the **action space**, the set of decisions available to the agent. A single **action** $a_t \in \mathcal{A}$ is the decision taken at time step t . The action space is *discrete* when it is a finite set of alternatives, and *continuous* when its elements are real valued, as is the case for the population size action used in this dissertation.
- $P(s_{t+1} \mid s_t, a_t)$ is the **transition function**, giving the probability of the environment moving to state s_{t+1} when action a_t is taken in state s_t .
- $R(s_t, a_t)$ is the **reward function**, producing the scalar **reward** r_t that represents the desirability of the resulting transition.
- $\gamma \in [0, 1]$ is the **discount factor**, which controls how much weight is placed on rewards received far in the future relative to immediate ones.

The defining assumption of this formalisation is the **Markov property**: the transition probability depends only on the current state and action, and not on the history of states and actions that preceded them. The state is therefore required to summarise everything about the past that is relevant to the future, which makes the choice of state representation a substantive design decision rather than a matter of convenience.

Interaction proceeds in **episodes**. At each time step the agent observes s_t , selects a_t , receives r_t , and the environment transitions to s_{t+1} ; the episode ends when a **terminal state** is reached. The quantity the agent seeks to maximise is the **return**, the discounted sum of future rewards from time t onwards:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

The agent’s behaviour is described by a **policy** $\pi(a \mid s)$, a mapping from states to a distribution over actions. A policy is *stochastic* when it assigns probabilities across several actions and *deterministic* when it assigns all probability to one; the distinction matters in this dissertation, since the same policy is sampled stochastically during training and evaluated deterministically (Section 4.4.2). The objective of learning is to find the policy that maximises the expected return:

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\pi} [G_t]$$

Two functions are used to estimate how good a situation is under a given policy. The **state value function** gives the expected return from a state, and the **action value function** gives the expected return from taking a particular action in that state:

$$V^{\pi}(s) = \mathbb{E}_{\pi} [G_t \mid s_t = s], \quad Q^{\pi}(s, a) = \mathbb{E}_{\pi} [G_t \mid s_t = s, a_t = a]$$

Their difference, the **advantage** $A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$, measures how much better a specific action is than the policy’s average behaviour in that state, and is central to the policy gradient methods discussed in Section 2.1.3.

A wide range of algorithmic approaches have been developed within this framework, including value based methods such as Q-learning, policy gradient methods, and actor-critic architectures, each offering a different trade off between sample efficiency, stability, and scalability [16]. A further distinction that recurs below is between *on-policy* algorithms, which learn only from experience generated by the policy currently being improved, and *off-policy* algorithms, which can reuse experience generated by earlier or different policies. This state-action-reward loop maps directly onto the problem of controlling an optimisation algorithm’s own hyperparameters at runtime, as described in Section 2.2.

2.1.2 Deep Reinforcement Learning

A significant limitation of traditional (tabular or linear) RL is its reliance on a tailored, low dimensional state representation. A tabular method stores one entry per state, or per state-action pair, which is only possible when $\mathcal{S}$ is finite and small. When the state space grows large or contains continuous components, as it does whenever a state feature is a real valued quantity, there are infinitely many distinct states and no entry can be visited often enough to estimate its value; tabular and linear methods therefore fail to generalise effectively across unseen states. Deep reinforcement learning (DRL) addresses this by combining RL with deep neural networks, which are used as **function approximators**: rather than storing V , Q or π explicitly, a network with parameters θ is trained to approximate them, so that experience gathered in one state informs the estimate at similar states it has never visited.

[19] demonstrated the practicality of this combination by training a convolutional neural network to play Atari 2600 games directly from raw pixel input, achieving human like performance using only the screen and the game score as input. This architecture, termed the Deep Q-Network (DQN), introduced two innovations that remain central to modern DRL. The first is **experience replay**, in which past transitions (s_t, a_t, r_t, s_{t+1}) are stored in a buffer and mini-batches are sampled from it uniformly at random for training, rather than learning from

each transition in the order it arrives; this breaks the temporal correlation between consecutive samples that destabilises online training [19]. The second is a separate **target network**, a periodically updated copy of the Q-network whose parameters are held fixed between updates, so that the regression target for the Q-value update does not shift at every step [20]. Together, these allowed the agent to learn a stable value function from the highly correlated, non stationary stream of data produced by its own interaction with the environment.

DQN and its value based descendants estimate $Q(s, a)$ for each available action and act greedily with respect to it, which requires enumerating the action space and so restricts them to discrete actions. Many control problems of interest, including the one addressed in this dissertation, have a continuous action space in which such enumeration is impossible without discretisation. This motivates the use of policy gradient and actor-critic methods, which parametrise a stochastic policy directly and can output continuous actions.

2.1.3 Proximal Policy Optimisation

Policy gradient methods optimise the parameters θ of a policy π_θ by ascending the gradient of the expected return with respect to θ directly, rather than deriving a policy from a learned value function. **Actor-critic** methods combine the two ideas: an *actor* network represents the policy $\pi_\theta(a | s)$ and selects actions, while a *critic* network estimates the value function $V(s)$ and is used to compute the advantage estimate $\hat{A}_t$ that tells the actor how much better than average each action taken was.

Proximal Policy Optimisation (PPO) [27] is an on-policy actor-critic algorithm that has become one of the most widely used DRL algorithms in practice, owing to its relative simplicity, stability, and strong empirical performance across both discrete and continuous control tasks. The difficulty PPO addresses is that a policy gradient step of the wrong size can move the policy far enough from the one that generated the current batch of experience that the batch no longer describes it, causing a collapse in performance from which learning may not recover. PPO controls this by defining the probability ratio between the updated and previous policies,

$$\rho_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$$

and optimising a clipped surrogate objective in which any update that moves this ratio beyond $1 \pm \epsilon$ receives no further improvement in the objective:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

This removes the incentive to make large policy changes on the basis of a single batch. It achieves the same broad purpose as a **trust region**, an explicit constraint bounding the divergence between successive policies, as enforced in the earlier TRPO algorithm, but without the computational overhead of solving a constrained optimisation problem at each update. In continuous action settings, the policy is typically parametrised as a Gaussian distribution whose mean and standard deviation are produced by a neural network, with actions sampled from this distribution during training and taken as the distribution mean at evaluation time.

PPO is a natural candidate for this dissertation for three reasons: it handles continuous action spaces directly, without the discretisation that a value based method such as DQN would require for a population size action; it is comparatively robust to hyperparameter choice, which matters given the expense of training on a black-box optimisation benchmark; and it has already been used successfully to control evolutionary algorithms in related work, including step size adaptation in CMA-ES [28] and evolution strategies more broadly [31]. The relationship between PPO and CMA-ES has also been explored in the opposite direction: [8]

observed that PPO’s exploration variance tends to shrink prematurely in continuous action tasks, and proposed PPO-CMA, which borrows CMA-ES’s approach to adaptively expanding exploration variance to improve PPO itself, rather than using PPO to control CMA-ES. This dissertation follows the direction of [28] and [31], using PPO as the controller and CMA-ES as the controlled system.

In particular, DRL agents built on algorithms such as PPO can learn adaptive policies over continuous parameter spaces, a property this dissertation exploits directly for configuring an optimisation algorithm’s hyperparameters dynamically during its own runtime.

2.2 Dynamic Algorithm Configuration

Optimisation algorithms typically require a set of **hyperparameters** that govern their behaviour, as distinct from the decision variables of the problem they are solving. A complete assignment of values to these hyperparameters is a **configuration**, and the algorithm being configured is referred to as the **target algorithm**. Configurations are conventionally chosen before execution and held fixed throughout the run, a practice known as **static configuration**. While a static configuration can be tuned per problem class, it is inherently biased towards the problems it was tuned on and often transfers poorly to other problems, or to other stages of the same optimisation run.

A further distinction that matters for both training and evaluation in this dissertation is between a problem and a **problem instance**. A benchmark function defines a family of related optimisation problems, and an instance is one concrete realisation of it, typically obtained by applying a randomised translation or rotation so that the location of the optimum differs while the structural character of the landscape is preserved. A configuration policy is trained over a distribution of instances rather than a single problem, and can then be evaluated on instances it has not seen, which is what distinguishes generalisation from memorisation.

Automated dynamic algorithm configuration (DAC) addresses the limitations of static configuration by treating hyperparameter control itself as a sequential decision making problem that runs alongside the target algorithm’s execution [1]. Rather than committing to a static configuration, a DAC controller observes the state of the algorithm (for example the progress of the search), the diversity of the current population, or features of the objective landscape encountered so far, and reconfigures one or more hyperparameters at each stage of the run.

This framing maps directly onto the MDP of Section 2.1.1, with each element of the tuple taking on a concrete meaning:

- the **state** s_t is a representation of the target algorithm’s internal condition and of the problem being solved at the point where a configuration decision is required;
- the **action** a_t is the value assigned to the hyperparameter under control, in this dissertation the population size λ ;
- the **transition** is one step of the target algorithm executed under that configuration, in this dissertation one CMA-ES generation;
- the **reward** r_t is derived from optimisation progress made over that step;
- an **episode** is a single run of the target algorithm on one problem instance, terminating when the algorithm stops or the budget is exhausted.

[1] formalise DAC approaches as a single MDP in this way and identify the key design choices this formalisation demands: the state representation of algorithm behaviour, the action

space over which configurations are selected, and the reward signal used to guide learning. Because DAC reduces to an MDP, any DRL algorithm capable of learning a policy over an MDP, including PPO, can in principle serve as the learning component of a DAC system.

DAC is especially compelling for algorithms whose optimal configuration varies substantially across problem instances, dimensionalities, or phases of the search process, a characteristic exhibited by CMA-ES, whose optimal population size is known to depend on the modality, ruggedness, and dimensionality of the objective function [10].

2.3 Evolutionary Algorithms

CMA-ES belongs to the broad family of evolutionary algorithms: population based, stochastic search-and-optimisation methods loosely inspired by biological evolution. The terminology is borrowed from that analogy. A **candidate solution**, or *individual*, is a single point in the search space; the **population** is the set of candidates held at one time; a **generation** is one complete cycle of producing, evaluating and selecting candidates; and the **fitness** of a candidate is the objective value the function under optimisation assigns to it. In the notation used throughout this dissertation, λ denotes the number of candidates sampled per generation and μ the number of them retained for producing the next generation.

2.3.1 Genetic Algorithms

Genetic algorithms (GA) maintain a population of candidate solutions across generations, selecting individuals for reproduction according to their fitness. Offspring are generated by subjecting parent candidates to variation operators such as mutation (modifying individual ‘genes’, or properties, of a candidate) and crossover (combining a subset of ‘genes’ from one candidate with those of another). The selection of parents for reproduction steers the population towards better regions of the search space [6]. Maintaining a population of candidates rather than a single solution implicitly allows exploration in parallel and helps the search escape local optima, which is important when optimising multimodal, noisy black-box functions.

2.3.2 Evolution Strategies

Evolution strategies (ES) are a class of evolutionary algorithms specialised for continuous parameter optimisation. They operate directly on real-valued vectors and rely primarily on Gaussian mutation for search [6]. Unlike a GA, a classical ES is typically self-adapting: the parameters that govern the mutation distribution, referred to as **strategy parameters** to distinguish them from the **object parameters** that encode the candidate solution itself, are subject to evolution alongside the candidates, allowing the algorithm to tune its own search behaviour online rather than relying on an externally fixed schedule.

2.3.3 CMA-ES

The problem setting is continuous black-box minimisation of an objective function $f : \mathbb{R}^n \rightarrow \mathbb{R}$, where n is the dimensionality of the search space. In the **black-box** setting the optimiser has no access to the analytical form of f , its gradient, or its shape, and can only query it: supplying a candidate $x \in \mathbb{R}^n$ and receiving the scalar $f(x)$ in return. The cost of a run is therefore measured in the number of such queries, or **function evaluations**, that it consumes.

CMA-ES is one of the most prominent and widely used evolution strategies for this setting. It maintains a multivariate Gaussian distribution over the search space, characterised by three quantities: the **mean** $m_t \in \mathbb{R}^n$, which is the current estimate of where the optimum lies; the

step size $\sigma_t \in \mathbb{R}_{>0}$, a scalar controlling the global scale of sampling; and the **covariance matrix** $C_t \in \mathbb{R}^{n \times n}$, which captures the shape and orientation of the distribution. At each generation t , λ candidates are sampled from this distribution:

$$x_i = m_t + \sigma_t y_i, \quad y_i \sim \mathcal{N}(0, C_t), \quad i = 1, \dots, \lambda$$

The candidates are evaluated and ranked, with $x_{i:\lambda}$ denoting the i -th best of them. The mean is then updated to a fitness weighted recombination of the μ best samples,

$$m_{t+1} = \sum_{i=1}^{\mu} w_i x_{i:\lambda}, \quad w_1 \geq \dots \geq w_{\mu} > 0, \quad \sum_{i=1}^{\mu} w_i = 1$$

where the w_i are the **recombination weights**, which give better ranked candidates greater influence over the new mean. Only the ranking of candidates is used, never their absolute fitness values, which is what makes CMA-ES invariant to monotonic transformations of f .

The covariance matrix is adapted to capture pairwise dependencies between variables, effectively learning the local curvature of the objective function [10]. Its update combines two components. The **rank- μ update** accumulates the outer products $y_{i:\lambda} y_{i:\lambda}^{\top}$ of the successful sampled directions in the current generation, and so estimates the covariance from the μ selected samples; the accuracy of this estimate depends directly on how many samples are available, and hence on λ , which is the connection [23] exploit to derive a population size adaptation rule. The rank-one update instead accumulates information across generations via an **evolution path**, an exponentially weighted running sum of the successive steps taken by the distribution mean. An evolution path is long when consecutive steps point in a consistent direction and short when they cancel each other out, which lets the algorithm distinguish sustained progress from random walking.

Step size is handled by a separate mechanism, **cumulative step size adaptation** (CSA), which compares the length of a second evolution path against the length that would be expected under random selection: a longer path indicates that steps are reinforcing one another and σ should grow, while a shorter one indicates over-stepping and σ should shrink. This prevents both premature convergence and excessively large steps [10].

Several properties of the objective function determine how difficult it is for this scheme. A function is **separable** if it can be optimised one variable at a time, and *non-separable* otherwise, in which case the dependencies between variables must be learned, which is precisely what C is for. A function is **ill-conditioned** when its sensitivity differs by orders of magnitude along different directions, producing long narrow valleys. A function is **unimodal** when it has a single optimum and **multimodal** when it has many local optima in addition to the global one, and is **rugged** when the landscape varies sharply over short distances. On multimodal and rugged landscapes an optimiser risks **premature convergence**: contracting onto a local optimum from which it cannot escape, since σ has by then become too small to sample outside its basin. A related failure is **stagnation**, where the search continues to consume evaluations without further improvement. Both are detected in practice by **termination criteria**, internal tests on the algorithm’s own state that decide when a run should stop, and both motivate **restart** schemes, which stop a stalled run and begin a new one with a different configuration (Section 3).

Despite CMA-ES’s strong default behaviour, several of its hyperparameters remain difficult to set well. Population size, λ , is among the most consequential. The default value is set as a simple function of search space dimensionality, $\lambda = 4 + \lfloor 3 \ln n \rfloor$ with $\mu = \lfloor \lambda/2 \rfloor$, and is calibrated for the well conditioned, unimodal case, so it can be markedly suboptimal on harder problem classes [10]. [14] provide early empirical evidence for this: evaluating the plain CMA evolution strategy on a multimodal test suite showed that a run with a small, default

sized population is prone to premature convergence on local optima, motivating the restart based schemes discussed in Section 3. On multimodal or noisy landscapes, larger populations improve exploration and reduce the risk of premature convergence, but at the cost of more function evaluations per generation, so an overly large population wastes computational budget on redundant candidates once the local neighbourhood has been adequately sampled. The optimal value of λ therefore depends on properties of the objective function that are, by definition, unknown in a black-box setting, making any single static configuration fundamentally limited.

This has motivated a substantial body of work on adapting CMA-ES’s configuration during a run. [30] showed that dynamically selecting among CMA-ES variants with different configurations, rather than committing to one for the whole run, can substantially outperform any single static configuration, particularly on problem instances with diverse landscapes. [28] demonstrated the same principle specifically for step size: a controller trained with RL across a distribution of problem instances outperformed CMA-ES’s own fixed step size adaptation rule on a diverse set of benchmark functions. This is an important proof of concept, establishing that RL based configuration of CMA-ES internals is not only feasible but effective for at least one hyperparameter. Separately, [25] showed that CMA-ES equipped with an adaptive learning rate can, with only the default population size, solve multimodal and noisy problems that would otherwise require a much larger population, a result that underlines how interconnected CMA-ES’s various hyperparameters are, and that population size is only one lever among several that jointly determine performance on hard landscapes.

2.4 Connecting the Parts

The preceding sections establish the three ingredients that jointly motivate this dissertation. DRL provides a general framework for learning adaptive policies from observations of a complex, evolving system [19], and PPO in particular provides a stable, well-understood algorithm for learning such a policy over a continuous action space [27]. DAC provides the conceptual bridge between RL and algorithm control, reframing a hyperparameter configuration problem as an MDP [1]. CMA-ES provides the target algorithm: a continuous black-box optimiser whose performance is highly sensitive to its population size, but whose optimal population size cannot be known in advance [10].

Together, this evidence suggests that RL based configuration of CMA-ES is both feasible and, on the related problem of step size adaptation, effective, and that a similar approach could plausibly be applied to population size. The next section surveys work that has already attempted to solve this specific problem with heuristic, rather than learned, adaptation rules, and identifies the gap this dissertation addresses.

3 Related Work

The population size parameter has a strong effect on CMA-ES performance, and no single static value is appropriate across all problems. This section surveys the work that has directly addressed dynamic population size control, ranging from deterministic restart based schemes to continuous, signal driven adaptation rules, and situates this dissertation’s DRL-based approach relative to that work.

3.1 IPOP-CMA-ES: Increasing Population [3]

The observation motivating IPOP-CMA-ES was that standard CMA-ES, run with a default λ , is liable to converge prematurely on multimodal functions - the search reaches a local

optimum and is unable to escape or explore other regions of the landscape, a failure mode documented empirically by [14]. IPOP counters this by restarting the algorithm and doubling λ every time a stagnation criterion is met. The progressive enlargement of the population across restarts means that later restarts search more globally, while early, cheap restarts with small populations exploit fast convergence on simpler parts of the landscape.

IPOP demonstrates strong empirical performance on the BBOB benchmark, and its success established two principles that inform all of the subsequent work surveyed below: population size is critical for balancing exploration and exploitation, and adapting it over the course of a run yields substantial gains over a fixed default. However, IPOP’s adaptation is entirely rule based and open loop. The decision to restart and double λ is triggered by a fixed stagnation criterion rather than by any learned or responsive assessment of the current search dynamics. The doubling schedule is also monotonically increasing: once the algorithm has committed to a larger population, it has no mechanism to recover the efficiency of a smaller one, even on a problem that did not require the increase. BIPOP-CMA-ES [9] partially addresses this rigidity by allocating the evaluation budget between two interleaved restart regimes: one with an increasing population, as in IPOP, and one that repeatedly restarts with a small population and a randomised initial step size. However, the underlying adaptation signal (a fixed stagnation criterion) remains identical.

3.2 AP-CMA-ES: Adaptive Population [2]

AP-CMA-ES embeds the adaptation mechanism within the ES itself, rather than relying on external restart logic. It augments the strategy with a self adaptive population size control scheme, in which λ is treated as an additional strategy parameter that is subject to evolution alongside the object parameters and is updated according to evolutionary pressure, allowing it to adapt dynamically to the fitness landscape encountered during search.

A key advantage over IPOP is bidirectionality: because the population can grow or shrink, the algorithm can reduce its sampling budget on simpler landscapes and increase it only when diversity is needed, improving overall function evaluation efficiency relative to IPOP. The authors demonstrate improved performance on a range of multimodal benchmark functions, particularly where the optimal population size varies across phases of the search. However, embedding population size adaptation within the ES itself introduces coupling between object level and strategy level search, making the resulting dynamics harder to analyse and tune. The self adaptive mechanism also relies on evolutionary selection pressure to drive changes in population size, which couples the rate of adaptation to the rate of selection rather than search state directly.

3.3 PSA-CMA-ES: Population Size Adaptation [24]

PSA-CMA-ES introduces an adaptation rule based on a success rate signal derived from an evolution path defined over the distribution parameters themselves. If the current population is producing sufficient improvement relative to a baseline, λ is reduced to improve efficiency; if improvement stalls, λ is increased to promote exploration. This built on an earlier proposal [23], which first framed the rank- μ update of CMA-ES as a stochastic natural gradient approach and introduced a measure of the accuracy of the current natural gradient estimate, using that accuracy measure to drive the population size adaptation. PSA-CMA-ES refines this into a lightweight rule, designed to integrate smoothly with CMA-ES’s existing covariance matrix and step size adaptation mechanisms.

PSA-CMA-ES achieves performance equivalent to or better than IPOP-CMA-ES on a range of BBOB functions, with the added benefit of bidirectionality, and represents a mature heuristic solution to the population size problem. Its main limitation is the success rate

signal itself - while informative, it is a relatively coarse characterisation of search progress. It captures whether improvement is occurring, but not information about the shape of the fitness landscape, the current covariance structure, or the likely future trajectory of the search. Because the heuristic is fixed at design time, it also cannot be improved through experience across problem instances, and performance on highly structured landscapes remains bounded by the expressiveness of the success rate signal alone.

3.4 CMAES-APOP: Adapting Population for Multimodal Functions [21]

CMAES-APOP introduces a population size adaptation mechanism designed specifically to detect when the algorithm is likely trapped in a local optimum, and to respond by increasing λ to promote escape and global exploration. The detection criterion is based on monitoring changes in the distribution median over successive generations; a stagnation signal indicates that the current population is insufficient to drive further progress and triggers an increase in λ .

CMAES-APOP is explicitly designed around the multimodal challenge, and benchmarking results on the BBOB noiseless testbed confirm competitive performance against IPOP and other restart strategies on functions with multiple local optima [22]. The heuristic remains tailored: the signals monitored (distribution median), the stagnation criteria, and the response triggered are all pre-determined, hence the mapping from internal state to population size is fixed. The thresholds can be set via automated configuration, but this would still be tuning the parameters of a fixed rule rather than learning the mapping itself.

3.5 Population Control Beyond Single-Objective CMA-ES

The idea that a growing population improves robustness at the cost of evaluation budget is not limited to the single objective setting. [17] investigated several static and dynamic strategies for increasing the population size of the multi objective variant, MO-CMA-ES, and found that a dynamic increasing population strategy significantly improves performance, particularly when the evaluation budget is small. While this dissertation is scoped to single objective BBOB functions, this result is a useful data point: it suggests that the underlying trade off this dissertation targets generalises beyond the specific single objective formulation of CMA-ES studied here, and that a learned controller of the kind proposed in this dissertation could plausibly extend to the multi objective setting in future work.

3.6 Research Gap

The work reviewed in this section reflects sustained and productive effort to address the population size problem in CMA-ES. The progression from IPOP’s simple restart-and-double scheme [3] to the success rate and distribution statistic heuristics of PSA-CMA-ES and CMAES-APOP [24, 21] represents a clear trajectory towards richer, more state aware adaptation. Across all these approaches, the structure of the adaptation rule is fixed at the design stage. These design rules disagree with each other on which signals are informative (e.g. restart triggers, evolutionary pressure, median stagnation), indicating that the choice is not settled. Moreover, these rules cannot update the mapping to population size decisions with experience across problem instances.

The DAC framework offers an alternative: learning a policy that maps algorithm states to population size decisions by training an RL agent across a variety of problem instances, rather than hand designing the mapping. This approach has already demonstrated viability in the closely related setting of step size adaptation in CMA-ES, where a learned controller

outperformed traditional heuristic methods across a diverse function benchmark [28]. Population size adaptation, however, presents a distinct challenge relative to step size that plausibly makes it harder to control. A change in λ potentially alters the quality of the covariance estimate, as opposed to step size, which merely rescales the next sampling step [10]. Moreover, λ sets the evaluation cost of subsequent generations, and an oversized population expends the limited budget on redundant candidates. This evaluation budget cannot be recovered, unlike step size, which can be corrected in the next generation. The policy must balance exploration against efficiency while reasoning about longer term consequences that a myopic rule has no mechanism to represent.

The application of DRL to this specific problem is further motivated by the capacity of deep neural networks to learn expressive, non-linear mappings from a rich state representation, in principle the full internal state of the CMA-ES distribution, its fitness history, and features of the landscape encountered so far, to a population size decision without the designer having to specify that mapping by hand. This dissertation implements and evaluates such a controller, trained with PPO, and situates its performance directly against the default configuration and IPOP-CMA-ES.

4 Methodology

This section describes the design of the population size controller: the formulation of the control problem as an MDP, the state representation and reward functions the policy is trained against, and the training and evaluation setup built around them. The software choices and the structure of the code that implements this design are described last, in Section 4.6.

4.1 Problem Formulation

The control problem instantiates the DAC formulation of Section 2.2, with CMA-ES as the target algorithm and the population size λ as the single hyperparameter under control. Concretely, the MDP $\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$ of Section 2.1.1 is instantiated as follows.

Transition. One environment step corresponds to exactly one CMA-ES generation. The controller is queried whenever CMA-ES requires a population size, the generation is then sampled, evaluated and used to update m , σ and C in the usual way, and the resulting internal state forms the next observation. Choosing the generation as the unit of time follows directly from the structure of the algorithm, since a generation is the smallest interval over which a change to λ can take effect at all: λ determines how many candidates are drawn, so it cannot be revised part way through drawing them. A coarser step, reconfiguring only every k generations as CMAES-APOP does (Section 3), would reduce the length of an episode and so make the credit assignment problem easier, but it would also impose a fixed reaction time on the policy that this dissertation has no principled basis for choosing.

Action. The action is the population size for the coming generation, a continuous scalar in $[2, \lambda_{\max}]$. The lower bound of 2 is the smallest population from which a recombination weighted mean update can be formed. The upper bound is a training tractability measure rather than a property of CMA-ES: since the number of function evaluations consumed by a generation is exactly λ , an unbounded action space would allow a single generation to consume an arbitrary share of the evaluation budget, and early in training, before the policy has learned anything, this would produce episodes that exhaust the budget in a handful of steps and so yield almost no transitions to learn from. Treating the action as continuous rather than discrete follows from the same reasoning that motivated PPO in Section 2.1.3: λ is an integer in practice, but the number of admissible values grows with $\lambda_{\max}$, and discretising the space finely enough to retain useful resolution would produce an action set too large for a

value based method to enumerate. The continuous action produced by the policy is therefore rounded to an integer population size at the point of use.

Episode. An episode is a single, complete run of CMA-ES on one problem instance, beginning at a fresh initialisation and ending when the true optimum is found, when a `modcma` termination criterion fires, or when the evaluation budget is exhausted. Episodes deliberately do not include a restart branch analogous to IPOP’s: a restart resets m , σ and C , which discards precisely the internal state the policy conditions on, so an episode spanning several restarts would contain transitions that violate the Markov assumption of Section 2.1.1 without the observation carrying any indication that a reset had occurred. The consequence is that the learned controller competes with IPOP under a structurally different budget policy, which is accounted for explicitly in the evaluation (Section 5.4).

State and **reward** are the two remaining design decisions, and are treated separately in Sections 4.2 and 4.3 below.

4.2 State Representation

Section 2.1.1 notes that the state is required to summarise everything about the past that is relevant to the future. In a DAC setting this requirement can never be met exactly: the full internal state of CMA-ES includes the $n \times n$ covariance matrix and the complete fitness history, which is both too large to present to a policy network at every generation and of a dimensionality that varies with the problem. The controller therefore receives an *observation*, a fixed size summary of that state, and the choice of what to include in it is the central state design decision.

One observation space, referred to as **PSB**, was implemented and used for all experiments in this dissertation. It comprises three scalar features.

Population size (λ_t). The current population size, normalised by $\lambda_{\max}$. This is included because the action is an absolute population size rather than an adjustment to the current one, so without it the policy cannot tell whether a given observation warrants an increase or a decrease. It also carries the immediate cost of the previous decision: since a generation consumes exactly λ evaluations, λ_t is the rate at which the run is currently spending its budget, and any trade off between exploration and efficiency has to be expressed in terms of it.

Step size (σ_t). The current global step size. Of the quantities CMA-ES maintains internally, σ is the most informative single scalar about which phase of the search the algorithm is in, because of how it is adapted: cumulative step size adaptation (Section 2.3.3) grows σ when successive steps reinforce one another and shrinks it when they cancel out, so a small or falling σ indicates a distribution contracting onto a basin, and a large or rising one indicates a search still moving across the landscape. That signal is directly relevant to the population size decision, since contraction onto a basin is exactly the situation in which a larger population is needed if the basin is a local optimum, and exactly the situation in which a smaller one suffices if it is not. It is also the natural quantity to include for continuity with existing work: σ is the hyperparameter that [28] learn to control, and PSA-CMA-ES derives its adaptation signal from an evolution path defined over the distribution parameters, of which σ is one [24].

Fraction of remaining budget. The number of evaluations consumed so far divided by the total budget allotted. A population size decision that is correct early in a run may be wrong late in it, since a large population only pays for itself if enough budget remains for the improved sampling to be exploited. Without this feature the observation contains no notion of how far through the run the algorithm is, and the policy cannot distinguish a situation where exploration is still affordable from one where the remaining budget should be spent exploiting what has already been found. It is also the feature that makes the state closest

to Markovian in the sense of Section 2.1.1, since the budget consumed is the only one of the three that is monotonic and therefore encodes the run’s position in time.

All three features are cheap: each is a scalar already maintained by `modcma`, so constructing the observation adds no function evaluations and negligible overhead to a generation. This is a deliberate constraint. A controller whose observations were expensive to compute would consume exactly the resource the optimisation is trying to conserve.

PSB omits several signals that the heuristics of Section 3 use directly. It carries no landscape features, no summary of the covariance structure, no fitness history beyond what the reward implicitly reflects, and no indication of which termination criteria are close to firing. The BBOB function ID and instance ID are available in the environment but are deliberately excluded from the observation, so that the policy is forced to generalise across functions and instances rather than learning to recognise them. PSB was chosen as a minimal starting point on the grounds that a policy which cannot improve on a static default from population size, step size and budget alone is unlikely to be rescued by a richer representation, and that establishing whether the minimal case works is a precondition for investing in a larger one. The results in Section 5.4 bear on how far that minimal representation goes, and richer observation spaces are the second item of future work in Section 6.

4.3 Reward Functions

The reward has to express a preference over complete runs, but is delivered one generation at a time, which is the central difficulty in designing it. What is ultimately wanted is that the run reaches the optimum using few evaluations; what can be observed at a single step is how much the best objective value improved and how many evaluations that improvement cost. Four reward functions were implemented, each making a different choice about how to turn the latter into a proxy for the former, and all four are tested in Section 5.3.

Every reward includes a bonus term: a positive scalar added only when the global optimum is found during an episode, and zero otherwise. This is the only part of the reward that refers to the actual objective of the task rather than to a proxy for it, and its magnitude therefore sets how strongly the policy weighs eventually succeeding against accumulating step by step progress. A set of magnitudes was tested empirically (Section 5.3); the value adopted after those experiments was 20.

1. **FBEST_IMP**: the raw improvement in the historic best objective value between the previous and current step,

$$r_t = f_{\text{best},t-1} - f_{\text{best},t} + \text{bonus}$$

This is the most direct expression of the thing being optimised, and serves as the baseline against which the other formulations are judged. Its weakness is one of scale. BBOB objective values differ by many orders of magnitude across the 24 functions and across the phases of a single run, so an improvement of a fixed numerical size means something quite different on one problem than on another. Since the policy is trained across all functions at once, the gradient signal is dominated by whichever problems happen to produce large absolute improvements, and the same behaviour receives wildly different reward depending on where in the suite it occurs.

2. **FBEST_IMP_RATIO**: the same improvement, expressed as a fraction of the previous best value,

$$r_t = \frac{f_{\text{best},t-1} - f_{\text{best},t}}{|f_{\text{best},t-1}|} + \text{bonus}$$

Dividing by the previous best rescales the improvement into a relative quantity, so that halving the distance to the optimum is worth the same whether the objective value is currently 10^6 or 10^{-3} . This directly addresses the scale problem above and makes reward comparable across functions and across the phases of a run, which matters because the policy is expected to generalise across both. The formulation is adapted from the improvement ratio rewards used in DACBench [7] and by [31]. Its known weakness is numerical: the denominator becomes small as the run approaches an optimum located near zero, which can produce unbounded rewards late in a successful episode.

3. **RAW_BEST**: the negative of the current best objective value,

$$r_t = -f_{\text{best},t} + \text{bonus}^3$$

This rewards the state the run is in rather than the progress made, and is included as a deliberate contrast to the two improvement based formulations. Since internal best values are positive under a minimisation objective, it accumulates negative reward at every step until the episode ends, which makes it susceptible to learning a policy that seeks to terminate the run as early as possible regardless of whether the optimum has been found: ending the episode is the one action reliably available for stopping the accumulation of negative reward, and early in training, before the policy can find optima, it is also the only one that works. The bonus is what is supposed to outweigh this, and this reward function is included primarily to test whether it does.

4. **FBEST_IMP_EVALS**: FBEST_IMP_RATIO further divided by the number of evaluations spent obtaining the improvement,

$$r_t = \frac{f_{\text{best},t-1} - f_{\text{best},t}}{|f_{\text{best},t-1}| \cdot \lambda} + \text{bonus}$$

Since a generation costs exactly λ evaluations, this expresses improvement per evaluation rather than improvement per generation, and so makes the efficiency half of the exploration and efficiency trade off explicit in the reward rather than leaving the policy to infer it. The intended effect is to penalise large populations that buy improvement at a disproportionate evaluation cost. The risk is the opposite bias: because the penalty applies at every step whereas the benefit of a large population is realised only occasionally, when it prevents a premature convergence that would otherwise have ended the run, the formulation may systematically undervalue exploration.

Two of these four normalise their own magnitude by construction, which is relevant to the choice not to apply reward normalisation during training (Section 4.4.3).

4.4 Training Setup

4.4.1 Problem Sampling

The environment is configured with a set of problems defined as the Cartesian product of supplied lists of BBOB function IDs, instance IDs, and dimensionalities. It cycles through this set rather than sampling from it at random, and reshuffles the ordering after every complete pass. This guarantees that every problem is seen an equal number of times, which a random sample would only achieve in expectation, while the reshuffling prevents the policy from being exposed to the same problem ordering on each pass and so learning anything specific to that sequence. Since each problem is revisited many times over the course of training, every `ioh` function object is constructed once at the start and reset before each pass rather than being reconstructed, which removes a substantial and entirely avoidable overhead from the inner loop.

4.4.2 Agent

The agent is derived directly from CleanRL’s continuous-action PPO implementation [15]. Both the actor and critic networks have three linear layers with tanh activations. All layers are initialised with orthogonal weight matrices, a practice introduced by OpenAI Baselines [5] and carried forward by CleanRL along with several other code level optimisations known to matter in practice for PPO [15]; orthogonal initialisation has been shown empirically to produce results equivalent to, or better than, other standard initialisation schemes for deep networks [18]. The critic’s output layer is initialised with a standard deviation of 1, since the value function can take a wide range of magnitudes, while the actor’s output layer is initialised to produce smaller values, since its output is used as the mean of a Gaussian action distribution and is subsequently rescaled into the environment’s action space.

The base CleanRL implementation only provides a method for sampling a stochastic action from the learned distribution, which is what training requires. A second method was added that instead returns the distribution’s mean deterministically. This is the distinction drawn in Section 2.1.1 between a stochastic and a deterministic policy, and both are needed here for different purposes: sampling during training is what produces the exploration the policy learns from, while at evaluation time sampling noise would make the same policy return different population sizes for identical states, so that repeated evaluation of one model would not be reproducible and any comparison against a deterministic baseline would be confounded by that variance.

The agent is initialised with the flattened dimensionality of the observation space as a constructor argument rather than a hard coded value, so that the same implementation can be reused unchanged if the observation space is later extended, which Section 6 identifies as a direction for future work.

4.4.3 Training Script

The training script follows the CleanRL implementation closely, with two substantive departures; its command line arguments were extended to accommodate the case specific inputs this dissertation requires, including the training problem set, the observation space, and the reward function.

The first departure is the removal of observation and reward normalisation. The original CleanRL implementation normalises both using a running estimate of their mean and variance, a design choice inherited from OpenAI Baselines [5] and intended to keep the input and reward scale roughly stationary over the course of training. This was removed for two reasons. The observation features described in Section 4.2 are already bounded and can be scaled directly within the environment, so a running estimate adds nothing but a source of non stationarity, since the same observation maps to a different network input depending on what has been seen before it. More importantly, two of the four reward functions normalise their own magnitude by construction, and applying a second, running normalisation on top of that would undo precisely the property the ratio based rewards were designed to have, making the reward comparisons in Section 5.3 a test of the running estimate rather than of the reward formulations themselves.

The second is the treatment of the action. The agent’s raw output falls broadly within $[-2, +2]$, the typical range of an untrained or lightly trained Gaussian policy head, and is rescaled linearly into $[2, \lambda_{\max}]$, followed by clipping to guarantee the result remains in range for unusually large sampled actions. A linear rescaling of a symmetric distribution is a questionable match for a strictly positive, right skewed quantity like population size, and Section 6 returns to this as a candidate explanation for some of the observed policy behaviour.

Training also uses Gymnasium’s `AsyncVectorEnv` to run multiple environment instances in parallel worker processes, reducing wall clock training time by collecting experience from several independent CMA-ES runs simultaneously. Since PPO is on-policy (Section 2.1.3), all experience in a batch must come from the current policy, so parallelism is the only means available of increasing the amount of experience collected per update.

4.5 Evaluation Setup

The learned policy is compared against two reference configurations: the `modcma` Default and IPOP-CMA-ES [3]. The Default configuration uses `modcma`’s built in population size for one complete, non restarting run, and represents the static configuration a user would obtain without intervening. IPOP starts from the same default but doubles λ and restarts CMA-ES every time a termination criterion is met, continuing until the evaluation budget is exhausted. Both are evaluated over the full protocol described in Section 5: all 24 BBOB noiseless functions, instances 0 to 14, at dimensions $\{2, 3, 5, 10, 20, 40\}$.

All three controllers, including the learned one, implement a single common interface and are executed through the same code path. A controller is required to implement a reset method, called at the start of a run, and a population size selection method, called whenever CMA-ES requires a new λ ; the two baselines are stateless, returning the current λ unchanged and double the current λ respectively, while the learned controller loads a set of trained weights, constructs the PSB observation from the state supplied to it, and queries the agent for a deterministic action. Routing all three through one interface means any difference in measured performance is attributable to the population size decisions themselves rather than to a difference in how the run around them was executed, which rules out a class of errors in which the learned controller is evaluated under subtly different conditions from the baselines it is compared against.

The information passed to a controller at each call is the same set from which the PSB observation is constructed, together with additional context that PSB does not use: the problem dimensionality and total budget, the internal state of the running CMA-ES instance, which termination criteria have triggered, the best fitness in the current generation and the best found so far, and, for BBOB problems, the function and instance IDs. Carrying more than the observation requires allows richer observation spaces to be constructed without changing the interface, and allows the unused fields to be logged.

4.6 Software and Libraries

All development and experiments used Python 3.10.19, since this version installed and ran reliably with both `modcma` and `ioh`. The initial choice was the current Python release (3.14.2), but several required libraries failed to install cleanly against it, so the project was moved to the older, more broadly supported 3.10 release.

4.6.1 CMA-ES Implementation

Three candidate CMA-ES implementations were evaluated: `pycma` [11], `cmaes` [26], and `modcma` [4]. `pycma` is the least flexible of the three: it is designed primarily to be used out-of-the-box to optimise a single given function. `cmaes` was also found to be less suitable: it is a compact, well documented library aimed at educational and demonstrative use rather than rigorous benchmarking, and likewise offers limited runtime control over population size. Neither exposes the internal strategy parameters needed to construct a controller state as cleanly as `modcma`, and neither was designed with dynamic reconfiguration in mind, making both unsuitable for this dissertation.

`modcma` takes a modular approach to CMA-ES, decomposing the algorithm into independently configurable components and exposing the parameters relevant to its internal mechanism directly, which can be read at each generation to construct the state passed to a population controller. It is used across several existing CMA-ES benchmarking studies [4], giving reasonable confidence in its correctness relative to a custom implementation, while still providing the runtime flexibility this dissertation requires.

4.6.2 RL Environment

DACBench [7] was the initial choice of RL environment, since it already provides RL ready environments for the dynamic configuration of several algorithms, including CMA-ES. In practice, two problems ruled it out. First, its CMA-ES environment formulates the MDP around step size adaptation, not population size, and adapting it to a population size action would have required reworking some of the environment’s internals. Second, we were unable to obtain reproducible test results under the seeding configuration we used. Repeated runs with the same seed did not reproduce the same sequence of problems, which would have undermined the reproducibility of the training experiments in Section 5. Given that the CMA-ES running functionality DACBench would otherwise have provided was already available from `modcma`, a custom built Gymnasium [29] environment was written instead. The design remains partially inspired by DACBench: its general approach to state representation, and the ratio based reward of Section 4.3, were adapted from DACBench’s conventions and modified for this dissertation’s action space.

4.6.3 Function Benchmarks

The COCO platform’s BBOB noiseless testbed [12, 13] was used as the set of evaluated black-box functions throughout. The initial plan was to use the reference `coco` library directly, but `IOHProfiler` [4] was adopted instead. It provides a substantially larger range of problem instances than the reference implementation, which matters directly here since the training and evaluation instance sets must be disjoint (Section 5), is better integrated with `modcma`, and additionally provides access to other black-box benchmark suites, such as CEC, should the environment need to be extended beyond BBOB in future work. `IOHProfiler`’s logging infrastructure also removed the need to write custom result collection code, and its companion analysis package, `iohinspector`, computes the AOCC and ECDF metrics used in Section 5 directly from its log format.

4.6.4 Code Structure

The implementation is organised into five modules, illustrated in Figure 1. `cmaes.py` hosts a **CMAES** class acting as both factory and runner: its `make_runner` method constructs a fresh **ModularCMAES** instance for a given function and configuration, used by the training environment on every reset, while a `__call__` method makes the class directly compatible with `ioh.Experiment` for evaluation, dispatching internally to either a restarting or a non restarting run function depending on the controller supplied. `controllers.py` defines the state dataclasses and the abstract **PopulationController** interface described in Section 4.5, together with the three concrete controllers **DefaultPop**, **IPOP** and **DRLPop**. `environments.py` implements the Gymnasium environment of Section 4.1, with observation and reward construction abstracted into dedicated methods so that new observation spaces and reward functions can be added without touching the environment loop. `agents.py` and `ppo.py` hold the agent definition and training script of Section 4.4. Evaluation runs and

logging are handled by `ioh.Experiment` and analysis by `iohinspector`, with example usage provided in `benchmark.py` and `results.py`.

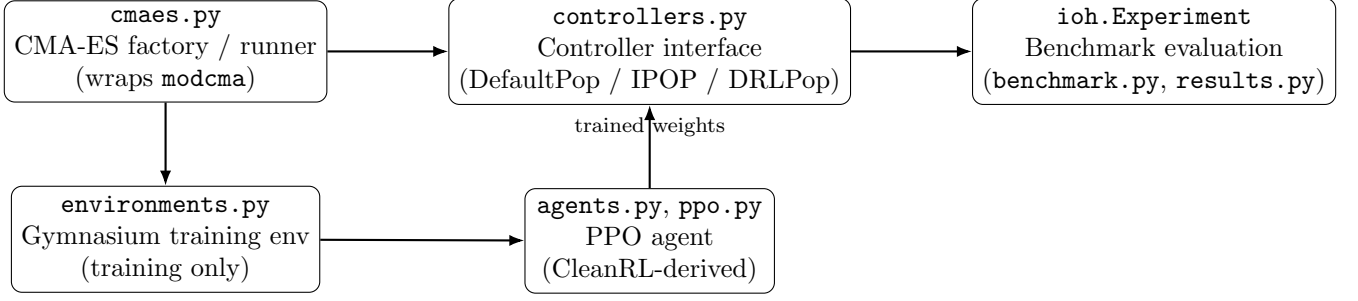


Figure 1: High-level system architecture

5 Experiments and Evaluation

Model training was conducted on all 24 BBOB noiseless functions [12], over 100 problem instances (15 to 114), and only at dimension 5, to keep the cost of the training experiments in Section 5.3 tractable. Evaluation, by contrast, always uses a disjoint set of unseen instances (0 to 14), so that reported performance reflects generalisation rather than memorisation of the training instances. Section 5.4 reports performance both on the training dimension (5) and on a set of held out dimensions $\{2, 3, 10, 20, 40\}$ that the policy never saw during training, to assess how well a policy trained at a single dimensionality generalises. The full evaluation protocol therefore covers $24 \text{ functions} \times 15 \text{ instances} \times 6 \text{ dimensions}$, giving 360 problems per dimension and 2160 problems in total for any single configuration.

5.1 Research Questions

The primary research objective is to determine whether a population size adaptation policy learned with DRL can compete with an established heuristic scheme. There are also the secondary motivations of analysing how this policy generalises over unseen problem configurations, and the stability of the training regime at larger training budgets. The research questions investigated and answered in this dissertation can be formalised as below:

1. **RQ1:** What reward function formulation allows a PPO agent to learn a useful population size adaptation policy for CMA-ES?
2. **RQ2:** Does more training data improve the learned policy, and is training stable at scale?
3. **RQ3:** Does the learned controller outperform a static default population size configuration?
4. **RQ4:** Does it match IPOPOP-CMA-ES, and does the comparison change once the evaluation budget consumed is accounted for?
5. **RQ5:** Does a policy trained on a single dimension generalise to unseen dimensions?

RQ1 and RQ2 are addressed by the training experiments in Section 5.3, RQ3 to RQ5 by the algorithm comparison in Section 5.4, and all five are drawn together in the critical appraisal in Section 5.5.

5.2 Metrics

Wall clock runtime was not used as a performance metric, since in a real deployment the objective function itself is expected to dominate total runtime far more than the optimiser’s own overhead; the number of objective function evaluations used is therefore the metric of primary interest, and all three of the metrics below are defined in terms of it.

The first and most important metric is the **raw success rate**: given a starting configuration, how often does the algorithm reach the global optimum before either exhausting its evaluation budget or triggering a `modcma` termination criterion. Termination criteria are used, rather than simply detecting the optimum directly, so that this metric reflects realistic deployment conditions, where an optimiser cannot know in advance whether it has found the true global optimum and must instead decide when to stop based only on its own internal state. Success rate is the prominent metric that we use to reason about all five research questions, however it has its own limitation: it is not budget aware. It completely discards the number of evaluations used and the rate of convergence, thus being biased in favour of IPOP, which will always completely exhaust the provided budget.

The need for budget awareness motivates the consideration of two additional supporting metrics: **area over the convergence curve** (AOCC), and **empirical cumulative distribution function** (ECDF), both computed with `iohinspector`. These two metrics are what allow RQ4 to be separated from RQ3: a controller can lose on raw success rate purely by stopping earlier, so the budget aware metrics are needed to establish whether IPOP is also the better policy per evaluation spent. AOCC is a single-number summary of how quickly an algorithm converges over the course of a run. It accumulates the normalised improvement in the objective value at every evaluation:

$$\text{AOCC} = \frac{1}{B} \sum_{t=1}^B \left(1 - \frac{\min(\max(\log(f_t - f^*), lb), ub) - lb}{ub - lb} \right)$$

where B is the total evaluation budget, f_t is the best objective value found up to evaluation t , and f^* is the known optimal value for the BBOB function being optimised. The constants lb and ub are the lower and upper bounds on the log scaled target precision, and serve to clip the accumulated term into $[0, 1]$: any precision better than lb contributes a full unit of area, and any precision worse than ub contributes none. An AOCC value closer to 1 indicates faster, more consistent convergence.

ECDF instead characterises how quickly a *set* of runs, taken together, reaches a target value. At a given evaluation budget b ,

$$\text{ECDF}(b) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}(e_i < b)$$

where $\mathbf{1}$ is the indicator function, e_i is the number of evaluations the i -th run took to reach its target precision relative to f^* , and n is the number of runs in the set. Runs that never reach the target do not contribute, which is why a curve can plateau below 1. An ECDF curve that rises earlier, and to a higher final value, indicates an algorithm that more reliably reaches its target using fewer evaluations, aggregated across many functions or instances at once.

5.3 Training Experiments

This subsection addresses the first two research questions: phases 1 to 3 formulate and test the reward functions (RQ1), and phase 4 attempts a more rigorous training experiment at a larger scale (RQ2). Several training experiments were run, varying aspects of the environment

and training hyperparameters, in order to identify a promising reward function and bonus magnitude before committing to the larger, more expensive comparison in Section 5.4. Each iterative model in this phase was assessed only by a preliminary success rate check and manual inspection of its learned policy and convergence behaviour, rather than the full evaluation protocol; the results are reported across four training phases below.

5.3.1 Phase 1

The first phase asks the broadest form of RQ1: of the four reward functions defined in Section 4.3, which of them produce a policy that solves problems at all. It trained for 10 million CMA-ES steps, batched over 32 parallel environments collecting 1024 steps each (305 batches total), with 10 epochs of optimisation per batch. All four reward functions were trained with a success bonus of 100. Table 1 reports the number of successes (out of 360 problems per dimension, or 2160 total per reward function) for each reward function and dimension.

FBEST_IMP and RAW_BEST lag well behind the other two reward functions in this phase. RAW_BEST in particular solves exactly 30 problems at every dimension except the largest, which is consistent with the failure mode anticipated in Section 4.3: since it accumulates negative reward at every step, the policy is rewarded for ending the run rather than for finding the optimum. FBEST_IMP_RATIO and FBEST_IMP_EVALS perform similarly to each other: FBEST_IMP_EVALS is slightly more successful on the training dimension (5), but FBEST_IMP_RATIO generalises marginally better, succeeding on more problems in total across dimensions. The two normalised rewards are therefore the only ones carried forward, and the remaining phases narrow the question from which reward function to how its bonus should be scaled.

Reward function	2	3	5	10	20	40	Sum
RAW_BEST	30	30	30	30	30	8	158
FBEST_IMP	48	44	35	33	43	15	218
FBEST_IMP_EVALS	204	183	178	145	108	28	846
FBEST_IMP_RATIO	209	187	174	150	118	32	870

Table 1: Successes for each reward function, by dimension, phase 1 (out of 360 per dimension).

5.3.2 Phase 2

Inspection of the phase 1 policies showed runs ending well before the evaluation budget was exhausted, so the second phase tested whether explicitly penalising early termination would push the policy towards longer, more productive runs. It repeated the phase 1 training budget and schedule, but trained only FBEST_IMP_RATIO and FBEST_IMP_EVALS, the two more promising reward functions from phase 1. A success bonus of 100 was again used, but a penalty of 30 was added for terminating without success when less than 90% of the evaluation budget had been used. Table 2 reports the results.

This penalty did not have the intended effect: performance collapsed relative to phase 1 for both reward functions, by roughly an order of magnitude. The most likely explanation is that the policy encountered the penalty far more often than the success bonus during early training, before it had learned to optimise effectively, so the penalty dominated the learning signal and discouraged exploration rather than encouraging longer, more productive runs. The hypothesis is therefore rejected, and the penalty term was removed from all subsequent phases.

Reward function	2	3	5	10	20	40	Sum
FBEST_IMP_EVALS	20	15	14	15	15	15	94
FBEST_IMP_RATIO	21	15	14	15	15	15	95

Table 2: Successes for each reward function, by dimension, phase 2 (out of 360 per dimension).

5.3.3 Phase 3

The third phase serves two purposes: confirming that the phase 2 collapse is attributable to the penalty rather than to some other change, and testing how sensitive the learned policy is to the magnitude of the success bonus. It again repeated the phase 1 training budget and schedule, training only FBEST_IMP_RATIO and FBEST_IMP_EVALS, this time with the early termination penalty removed and the success bonus reduced from 100 to 20. Table 3 reports the results.

Both reward functions again produce similar overall results, and both recover fully from the phase 2 collapse, which confirms the penalty as its cause. FBEST_IMP_EVALS is again slightly more successful on the training dimension, while FBEST_IMP_RATIO is again marginally stronger in total, and both are close to their phase 1 performance, with the softer bonus producing a small, consistent decrease relative to phase 1 for both reward functions. The bonus magnitude therefore matters, but far less than the presence or absence of the penalty; the smaller bonus of 20 was retained for phase 4 on the grounds that it produces comparable performance with a less extreme reward scale.

Reward function	2	3	5	10	20	40	Sum
FBEST_IMP_EVALS	207	183	173	147	106	26	842
FBEST_IMP_RATIO	212	179	159	148	112	33	843

Table 3: Successes for each reward function, by dimension, phase 3 (out of 360 per dimension).

5.3.4 Phase 4

The first three phases all used the same training budget, so none of them can say whether the policies were still improving when training stopped. The fourth phase addresses RQ2 directly, by training for substantially longer and checkpointing throughout, so that policy performance can be observed as a function of the amount of training data seen rather than only at the end. It used a training budget of 100 million CMA-ES steps, with 64 parallel environments (double the previous phases), checkpointing FBEST_IMP_RATIO and FBEST_IMP_EVALS every 50 batches. Both reward functions again used a success bonus of 20. Training on FBEST_IMP_RATIO terminated early, at 750 batches, due to a NaN error in the optimisation; the complete checkpoint data for both reward functions is given in Table 6 in Appendix B. Figures 2 and 3 plot success rate against training batches for FBEST_IMP_EVALS and FBEST_IMP_RATIO respectively.

FBEST_IMP_EVALS collapses in performance by around 100 training batches and remains low for the rest of training, with success rates staying slightly higher in the generalised case than on dimension 5 alone. That the aggregate rate exceeds the dimension 5 rate is itself informative: a policy that had learned anything dimension specific would be expected to do best where it was trained, so the ordering is consistent with the collapsed policy settling on a near constant population size that happens to suit the cheaper, lower dimensional problems. FBEST_IMP_RATIO behaves very differently, and much more favourably: performance peaks

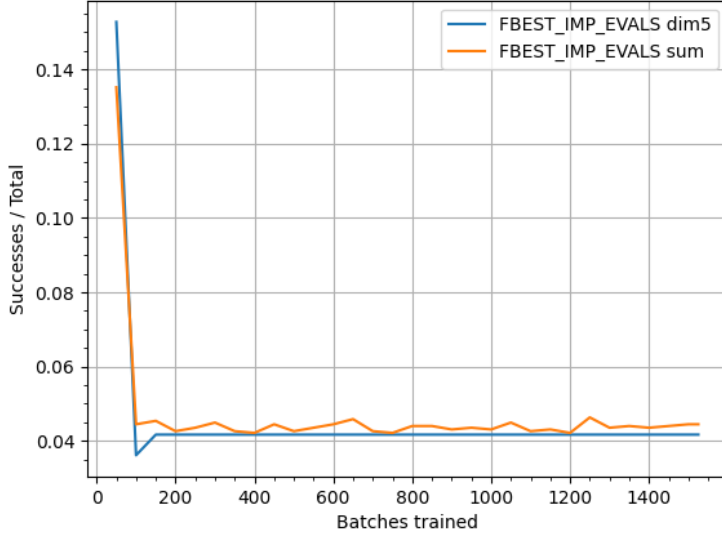


Figure 2: Success rate against training batches for FBEST_IMP_EVALS, phase 4.

around 150 training batches in both the dimension 5 and aggregate cases, remains consistently higher on dimension 5 than in aggregate, and stays broadly flat afterwards, with a small further improvement visible near the point where training stopped. The answer to RQ2 is therefore a qualified one: more training data does not monotonically improve the policy, the best checkpoint arrives early and the curve is flat thereafter, and training was not stable enough to run the configuration to its full budget. The small upturn at the end suggests that further gains might be available from a longer training run with a more carefully tuned learning rate schedule, an avenue discussed further in Section 6.

5.4 Algorithm Comparison

The final evaluation addresses RQ3, RQ4 and RQ5, comparing one selected DRL policy against the Default and IPOP-CMA-ES baselines (Section 4.5) across all three metrics from Section 5.2. The policy selected for this comparison is the FBEST_IMP_RATIO model from phase 1, chosen because it had the best success rate of any model trained across the first three phases. The phase 4 model trained on the same reward function performed marginally better at its peak (150 batches), but phase 4 was added later in the project specifically to study the effect of longer training, and is excluded from this comparison to avoid conflating the effect of the reward function with the effect of the larger training run: 150 batches under phase 4’s 64-environment configuration corresponds to approximately 300 batches’ worth of environment steps under the phase 1 to 3 configuration, so the phase 1 model has seen a comparable amount of training data to the phase 4 model at its peak.

One caveat applies throughout this comparison. The model was selected on its success rates over the evaluation instances, which are the same instances the comparison then reports, so the DRLPop success rates in Table 4 are identical to the FBEST_IMP_RATIO row of Table 1 and are a selection optimistic estimate rather than a fully held out one. Selection was made on success rate alone, so the AOCC and ECDF results reported below are not affected in the same way and are the less biased of the three comparisons.

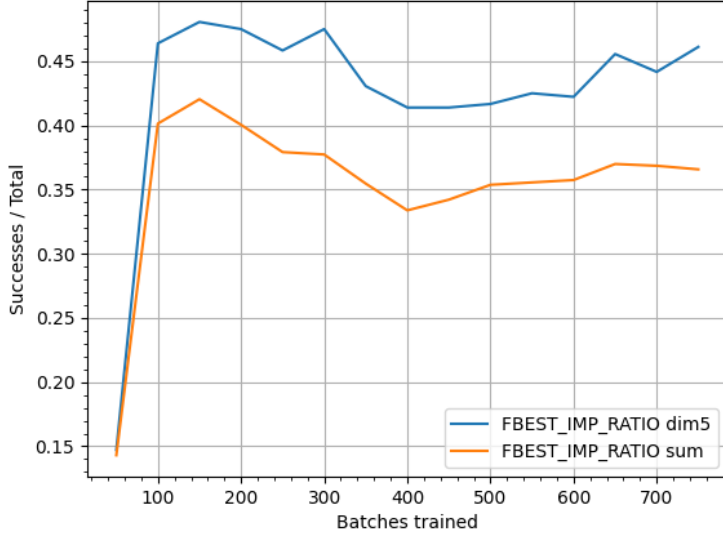


Figure 3: Success rate against training batches for FBEST_IMP_RATIO, phase 4.

5.4.1 Success Rates

Table 4 reports the number of successes for each controller across the full evaluation protocol (2160 problems per controller). The DRL controller (**DRLPop**) roughly doubles the success rate of the static Default configuration, which answers RQ3 affirmatively, but a full-budget IPOP evaluation achieves marginally more successes than DRLPop. This is broadly expected: Default and DRLPop both terminate as soon as their own termination criteria fire, using only a fraction of the available budget, whereas IPOP restarts repeatedly and continues consuming budget until it is exhausted. Table 5 quantifies this, with IPOP using essentially the entire allotted budget on every dimension while DRLPop uses under half of it. We treat this as a genuine trade off between the number of evaluations expended and the final success rate achieved, rather than a straightforward win for IPOP, and examine it further via AOCC and ECDF below, both of which are budget aware.

Controller	2	3	5	10	20	40	Sum
DRLPop	209	187	174	150	118	32	870
Default	152	125	81	73	20	15	466
IPOP	249	207	193	173	123	41	986

Table 4: Successes for each controller, by dimension (out of 360 per dimension).

Controller	2	3	5	10	20	40	avg
DRLPop	4219	4050	3653	3476	3723	4408	3922
Default	399	495	521	708	998	1753	812
IPOP	10028	10023	10020	10008	10005	10003	10015

Table 5: Mean evaluations per dimension used by each controller, by dimension (total allotted $d \cdot 10000$ per problem).

5.4.2 AOCC

AOCC values were computed up to the maximum allotted budget on each dimension separately. Full, per function results for dimension 5 and averaged across all six dimensions are given in Tables 7 and 8 (Appendix B). At dimension 5, DRLPop achieves the best AOCC on 3 of the 24 functions, Default on 6, and IPOP on the remaining 15. DRLPop outperforms Default on 11 of the 24 functions, 8 of them among the multimodal functions (BBOB functions 15 to 24), and has a higher average AOCC than Default at dimension 5 (0.761 vs. 0.733), despite winning on fewer individual functions, since its margin of improvement on the functions where it wins is larger than its margin of loss elsewhere.

Averaged across all six dimensions, DRLPop is the best controller on only one function, but still beats Default on 12 of the 24 functions and remains ahead of Default on average (0.727 vs. 0.715); IPOP remains the strongest controller by AOCC in both cases (0.823 at dimension 5, 0.801 averaged across dimensions). Accounting for budget therefore does not overturn the success rate result of RQ4: IPOP is ahead of DRLPop on a metric that charges it for every evaluation it spends.

5.4.3 ECDF

Figure 4 plots the aggregated ECDF curve for all three controllers; per function curves for all 24 BBOB functions are given in Figures 5 to 7 (Appendix B).

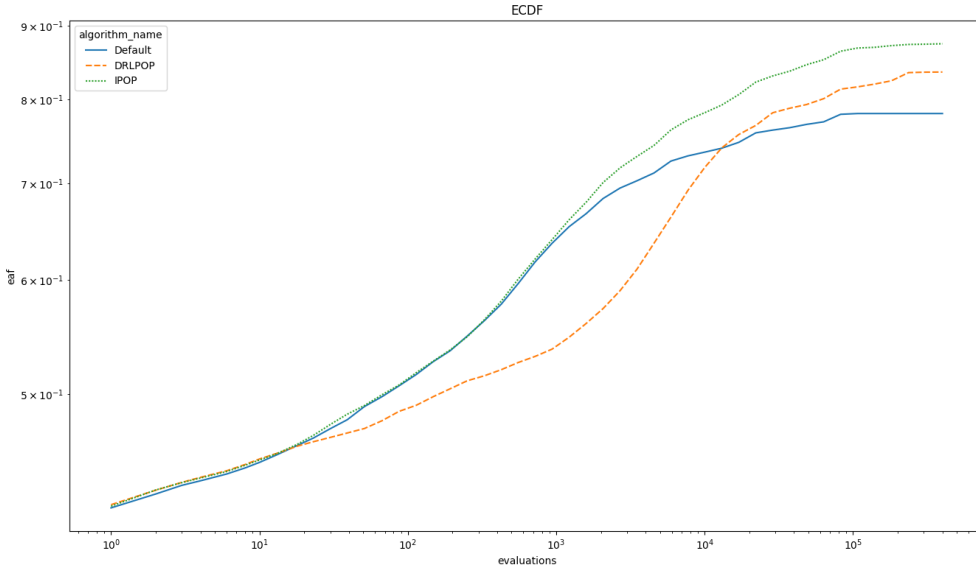


Figure 4: Aggregated ECDF across all three population controllers.

The aggregated curve shows DRLPop climbing more slowly than Default and IPOP over the first roughly 100 evaluations, before increasing sharply over the 1,000 to 10,000 evaluation range and ultimately reaching a higher final fraction of completed optimisations than Default. The per function curves show a broadly consistent pattern: DRLPop starts slower, then accelerates and, on several functions, ends close to or matching IPOP’s final completion rate, though the point at which DRLPop overtakes Default varies considerably by function, and on a handful of functions (visible in Figures 5–7) DRLPop’s curve plateaus below both baselines, indicating it fails to make further progress once its own termination criteria fire. The slow

start is what would be expected of a policy that samples a larger population early on, since each generation then costs more evaluations before any target is reached; the later acceleration is where that larger sample begins to pay for itself.

5.4.4 Generalisation Across Dimensions

RQ5 asks whether a policy trained only at dimension 5 transfers to dimensionalities it never saw. The dimension wise columns of Table 4 answer this in the negative. DRLPop solves 209 of 360 problems at dimension 2, but only 32 of 360 at dimension 40, a degradation of roughly a factor of six across the range tested. The same pattern is visible in every training phase (Tables 1 and 3), so it is a property of the training regime rather than of any single model.

Two points qualify this. First, all three controllers degrade with dimensionality, since higher dimensional BBOB problems are simply harder at a fixed budget per dimension; Default falls from 152 to 15 and IPOP from 249 to 41 over the same range. DRLPop’s degradation is therefore not solely a generalisation failure. Second, DRLPop’s performance relative to IPOP does hold up reasonably well in the middle of the range and is closest at dimension 20 (118 against 123), so the transfer is not uniformly poor. What the results do show is that the policy is at its weakest exactly where the population size decision matters most, at dimension 40, and that training on a single dimensionality is a plausible cause. Training across a mixture of dimensionalities is accordingly the first item of future work in Section 6.

5.5 Critical Appraisal

Taken together, the results support a specific, moderate conclusion rather than a strong one in either direction, and each of the five research questions can now be answered.

On RQ1, a reward based on the ratio of fitness improvement to the previous best value (FBEST_IMP_RATIO) was consistently the strongest or joint strongest choice across all four phases. The two normalised rewards were the only ones that produced a usable policy at all, which is the clearest single result of the training experiments: the raw improvement and raw best value rewards both failed, and they failed in the ways anticipated when they were defined. On RQ2 the answer is qualified, as set out in phase 4: more training data does not reliably help beyond an early peak, and the most promising configuration could not be run to its full budget.

On RQ3, the learned controller clearly improves on the default, roughly doubling its success rate and beating it on both AOCC averages, though the current evidence does not separate a state dependent strategy from a systematically larger population. Distinguishing the two would require logging the population sizes selected over the course of a run and comparing against a static configuration matched to DRLPop’s mean λ , which the existing pipeline supports but which was not carried out here. The improvement is concentrated where it should plausibly be expected: DRLPop’s largest gains over Default are on the multimodal BBOB functions (15–24), the exact class of problem that motivated IPOP and its successors in the first place.

On RQ4, DRLPop does not match IPOP-CMA-ES on this benchmark, on any of the three metrics. Part of this gap is structural rather than a failure of learning: IPOP is permitted to keep restarting and consuming budget for as long as the run continues, while DRLPop and Default both stop as soon as their own termination criteria fire, so IPOP’s advantage in raw success rate is partly a consequence of spending more evaluations, not necessarily of a better policy per evaluation spent. Table 5 puts that difference at roughly two and a half times more evaluations for a 13% higher success rate. The AOCC and ECDF results, which are budget aware, narrow this gap but do not close it: IPOP remains the strongest controller by average AOCC in every configuration tested. A second, and more limiting, part of the gap

is a genuine limitation of the current implementation - the policy used for the head-to-head comparison was trained on a single dimension (5) and a single observation space (PSB). A policy exposed to a wider range of training dimensionalities, or a richer observation space incorporating landscape features beyond population size, step size, and budget, would be needed before drawing a stronger conclusion about the ceiling of this approach relative to IPOP.

On RQ5, Section 5.4.4 shows performance degrading substantially as the evaluation dimensionality moves away from the training dimensionality, from 209/360 successes at dimension 2 down to 32/360 at dimension 40. Some of that decline is intrinsic to the benchmark rather than to the policy, since both baselines degrade too, but the effect is large enough that no strong claim about transfer can be made from a single dimension policy.

The training experiments in Section 5.3 also surface two reliability concerns that a future iteration of this work should address directly. First, reward design is fragile: the early termination penalty introduced in phase 2 collapsed performance by roughly an order of magnitude relative to phase 1 and phase 3, despite being intended to improve the policy’s behaviour, and the bonus magnitude alone (100 versus 20) produced a small but consistent difference in outcome between phase 1 and phase 3. Second, training stability is not guaranteed at scale: the phase 4 FBEST_IMP_RATIO run, the single most promising configuration identified in this dissertation, terminated early due to a NaN error before its longer training budget could be fully used. Both observations suggest that the reward function and training stability of this approach still need further engineering before it could be considered a reliable, drop-in replacement for a heuristic controller such as IPOP.

6 Future Work

Several directions follow naturally from the results and limitations identified in Section 5.5.

Training across dimensions. All models in this dissertation were trained at a single dimensionality (5) and evaluated for generalisation to five held out dimensionalities. Performance degraded substantially as the evaluation dimensionality moved away from the training dimensionality (Table 4), suggesting that training directly on a mixture of dimensionalities, rather than only evaluating generalisation after the fact, is a natural next step and may close some of the remaining gap to IPOP at higher dimensions.

Richer observation spaces. Only one observation space (PSB: population, step size, and scaled budget) was implemented and tested. Extending the state representation with additional CMA-ES internals already exposed by `modcma` would give the policy access to the kind of landscape and search progress information that the strongest existing heuristics, such as PSA-CMA-ES and CMAES-APOP, rely on directly (Section 3), and is a natural extension given the environment was already designed to support multiple, swappable observation spaces.

Action distribution. The agent currently produces population size actions from a Gaussian distribution, subsequently scaled and clipped into a valid range. Inspecting the actions produced during training and evaluation showed a strong clustering in the 30 to 90 range, and an action distribution that is symmetric on the (unclipped) real line is a poor match for a strictly positive, right skewed quantity like population size: extreme upward corrections are clipped less aggressively than a log scaled action would allow, which may explain why CMA-ES sometimes takes a large number of generations to converge on a gentler, more gradual population curve, and can be susceptible to a locally convincing but suboptimal region of the landscape. There may be merit in testing agents that instead produce actions from a log-normal distribution (always positive, and thicker tailed than a Gaussian), or that treat

the action as a multiplicative or exponential transformation of the current population size rather than an absolute value.

Longer, more stable training. The most promising configuration identified (FBEST_IMP_RATIO with a bonus of 20) showed a further, small improvement near the end of its training budget in phase 4 (Section 5.3) before that run terminated early due to numerical instability. Diagnosing and fixing the cause of the NaN error (candidates include gradient clipping, the learning rate schedule, or numerical issues in the ratio based reward when $f_{\text{best},t-1}$ is close to zero) and rerunning this configuration to a larger training budget is a direct, low risk extension of the existing pipeline that could plausibly narrow the remaining gap to IPOP.

Extending the baseline set. The comparison in Section 5.4 evaluates against Default and IPOP-CMA-ES only. The introduction (Section 3) also identifies BIPOP-CMA-ES [9], PSA-CMA-ES [24], and CMAES-APOP [21] as strong, more state aware heuristic baselines; the benchmarking pipeline described in Section 4.5 was designed around a common **PopulationController** interface specifically so that further baselines could be added with comparatively little additional code, and doing so would give a fuller picture of where a learned controller sits relative to the strongest existing heuristics, rather than only the earliest one.

7 Conclusion

This dissertation investigated whether a population size controller for CMA-ES, trained with deep reinforcement learning, can outperform existing heuristic approaches such as IPOP-CMA-ES. A custom Gymnasium environment and PPO-based training pipeline, built around `modcma` and `IOHProfiler`, were used to train and evaluate policies across four candidate reward functions on the BBOB noiseless testbed.

The central empirical finding is a mixed one: the learned controller (DRLPop) clearly and substantially outperforms a static default population size, roughly doubling its success rate and beating it on average AOCC at every dimensionality tested, with its largest gains concentrated on multimodal functions, the class of problem that population size adaptation is specifically intended to help with. At the same time, DRLPop does not outperform IPOP-CMA-ES on this benchmark, on any of the three metrics used, though part of that gap reflects IPOP’s willingness to spend a larger evaluation budget rather than a straightforwardly better policy. Among the reward functions tested, one based on the ratio of fitness improvement to the previous best value (FBEST_IMP_RATIO) was consistently the strongest or joint strongest choice across all four training phases, and is the most promising direction for further work.

Overall, this dissertation does not find strong evidence that a PPO based population size controller, in its current form, outperforms the strongest existing heuristic for this problem. It does find evidence that such a controller is a viable, learnable improvement over a static default, and identifies a specific, promising reward function and a concrete set of extensions: multi-dimension training, a richer observation space, and further baselines, that could plausibly close the remaining gap to IPOP in future work.

References

- [1] ADRIAENSEN, S., BIEDENKAPP, A., SHALA, G., AWAD, N., EIMER, T., LINDAUER, M., AND HUTTER, F. Automated dynamic algorithm configuration. *Journal of Artificial Intelligence Research* 75 (2022), 1633–1699.
- [2] AHRARI, A., AND SHARIAT-PANAHI, M. An improved evolution strategy with adaptive population size. *Optimization* 64, 12 (2015), 2567–2586.
- [3] AUGER, A., AND HANSEN, N. A restart cma evolution strategy with increasing population size. In *2005 IEEE congress on evolutionary computation* (2005), vol. 2, IEEE, pp. 1769–1776.
- [4] DE NOBEL, J., VERMETTEN, D., WANG, H., DOERR, C., AND BÄCK, T. Tuning as a means of assessing the benefits of new ideas in interplay with existing algorithmic modules. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion* (2021), pp. 1375–1384.
- [5] DHARIWAL, P., HESSE, C., KLIMOV, O., NICHOL, A., PLAPPERT, M., RADFORD, A., SCHULMAN, J., SIDOR, S., WU, Y., AND ZHOKHOV, P. Openai baselines. <https://github.com/openai/baselines>, 2017.
- [6] DIANATI, M., SONG, I., AND TREIBER, M. An introduction to genetic algorithms and evolution strategies. Tech. rep., Technical report, University of Waterloo, Ontario, N2L 3G1, Canada, 2002.
- [7] EIMER, T., BIEDENKAPP, A., REIMER, M., ADRIAENSEN, S., HUTTER, F., AND LINDAUER, M. Dacbench: A benchmark library for dynamic algorithm configuration. *arXiv preprint arXiv:2105.08541* (2021).
- [8] HÄMÄLÄINEN, P., BABADI, A., MA, X., AND LEHTINEN, J. Ppo-cma: Proximal policy optimization with covariance matrix adaptation. In *2020 IEEE 30th International Workshop on Machine Learning for Signal Processing (MLSP)* (2020), IEEE, pp. 1–6.
- [9] HANSEN, N. Benchmarking a bi-population cma-es on the bbob-2009 function testbed. In *Proceedings of the 11th annual conference companion on genetic and evolutionary computation conference: late breaking papers* (2009), pp. 2389–2396.
- [10] HANSEN, N. The cma evolution strategy: A tutorial. *arXiv preprint arXiv:1604.00772* (2016).
- [11] HANSEN, N., AKIMOTO, Y., AND BAUDIS, P. CMA-ES/pycma on Github. Zenodo, DOI:10.5281/zenodo.2559634, Feb. 2019.
- [12] HANSEN, N., AUGER, A., ROS, R., MERSMANN, O., TUŠAR, T., AND BROCKHOFF, D. Coco: A platform for comparing continuous optimizers in a black-box setting. *Optimization Methods and Software* 36, 1 (2021), 114–144.
- [13] HANSEN, N., FINCK, S., ROS, R., AND AUGER, A. *Real-parameter black-box optimization benchmarking 2009: Noiseless functions definitions*. PhD thesis, INRIA, 2009.
- [14] HANSEN, N., AND KERN, S. Evaluating the cma evolution strategy on multimodal test functions. In *International conference on parallel problem solving from nature* (2004), Springer, pp. 282–291.

- [15] HUANG, S., DOSSA, R. F. J., YE, C., BRAGA, J., CHAKRABORTY, D., MEHTA, K., AND ARAŖSJO, J. G. Cleanrl: High-quality single-file implementations of deep reinforcement learning algorithms. *Journal of Machine Learning Research* 23, 274 (2022), 1–18.
- [16] KAEHLING, L. P., LITTMAN, M. L., AND MOORE, A. W. Reinforcement learning: A survey. *Journal of artificial intelligence research* 4 (1996), 237–285.
- [17] LIMMER, S., AND FEY, D. Investigation of strategies for an increasing population size in multi-objective cma-es. In *2016 IEEE Congress on Evolutionary Computation (CEC)* (2016), IEEE, pp. 476–483.
- [18] MISHKIN, D., AND MATAS, J. All you need is a good init. *arXiv preprint arXiv:1511.06422* (2015).
- [19] MNIH, V., KAVUKCUOGLU, K., SILVER, D., GRAVES, A., ANTONOGLOU, I., WIERSTRA, D., AND RIEDMILLER, M. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602* (2013).
- [20] MNIH, V., KAVUKCUOGLU, K., SILVER, D., RUSU, A. A., VENESS, J., BELLEMARE, M. G., GRAVES, A., RIEDMILLER, M., FIDJELAND, A. K., OSTROVSKI, G., ET AL. Human-level control through deep reinforcement learning. *nature* 518, 7540 (2015), 529–533.
- [21] NGUYEN, D. M. An adapting population size approach in the cma-es for multimodal functions. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion* (2018), pp. 219–220.
- [22] NGUYEN, D. M., AND HANSEN, N. Benchmarking cmaes-apop on the bbob noiseless testbed. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion* (2017), pp. 1756–1763.
- [23] NISHIDA, K., AND AKIMOTO, Y. Population size adaptation for the cma-es based on the estimation accuracy of the natural gradient. In *Proceedings of the Genetic and Evolutionary Computation Conference 2016* (2016), pp. 237–244.
- [24] NISHIDA, K., AND AKIMOTO, Y. Psa-cma-es: Cma-es with population size adaptation. In *Proceedings of the Genetic and Evolutionary Computation Conference* (2018), pp. 865–872.
- [25] NOMURA, M., AKIMOTO, Y., AND ONO, I. Cma-es with learning rate adaptation: Can cma-es with default population size solve multimodal and noisy problems? In *Proceedings of the Genetic and Evolutionary Computation Conference* (2023), pp. 839–847.
- [26] NOMURA, M., SHIBATA, M., AND HAMANO, R. cmaes: A simple yet practical python library for cma-es. *arXiv preprint arXiv:2402.01373* (2024).
- [27] SCHULMAN, J., WOLSKI, F., DHARIWAL, P., RADFORD, A., AND KLIMOV, O. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017).
- [28] SHALA, G., BIEDENKAPP, A., AWAD, N., ADRIAENSEN, S., LINDAUER, M., AND HUTTER, F. Learning step-size adaptation in cma-es. In *International conference on parallel problem solving from nature* (2020), Springer, pp. 691–706.

- [29] TOWERS, M., KWIATKOWSKI, A., BALIS, J., DE COLA, G., DELEU, T., GOULÃO, M., ANDREAS, K., KRIMMEL, M., KG, A., PEREZ-VICENTE, R., ET AL. Gymnasium: A standard interface for reinforcement learning environments. *Advances in Neural Information Processing Systems 38* (2026).
- [30] VERMETTEN, D., VAN RIJN, S., BÄCK, T., AND DOERR, C. Online selection of cma-es variants. In *Proceedings of the Genetic and Evolutionary Computation Conference* (2019), pp. 951–959.
- [31] XU, T., CHEN, H. C., AND HE, J. Accelerate evolution strategy by proximal policy optimization. In *Proceedings of the Genetic and Evolutionary Computation Conference* (2024), pp. 1064–1072.

A User Manual

This appendix describes how to install and run the system described in Section 4. All commands assume a Unix-like shell and the submitted source directory as the working directory. The full code and models trained and evaluated in this dissertation can be found at <https://github.com/Phoenix-the-4th/DRLPop-CMAES/tree/main>. This repository is private at the time of writing this dissertation, but will be made public as soon as the school regulations allow.

A.1 Environment Setup

The project requires Python 3.10. The submitted archive contains eight files at the top level: the five modules described in Section 4.6 (`cmaes.py`, `controllers.py`, `environments.py`, `agents.py`, `ppo.py`), the two example scripts (`benchmark.py`, `results.py`), and `requirements.txt`.

```
python3.10 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

`requirements.txt` is a full freeze of the environment used for the reported experiments and includes CUDA builds of PyTorch. On a machine without an NVIDIA GPU the CUDA packages can be omitted and a CPU build of `torch` installed instead; all code selects the device at runtime via `torch.cuda.is_available()` and runs unchanged on CPU. The packages the system actually depends on are `modcma` 1.2.0, `ioh` 0.3.22, `iohinspector` 0.0.8, `gymnasium` 1.3.0, `torch` 2.13.0, `tyro`, `tensorboard`, `wandb`, `numpy`, `pandas`, `polars` and `matplotlib`.

A.2 Training a Policy

Training is launched through `ppo.py`, whose command line interface is generated by `tyro` from the `Args` dataclass. Every field of that dataclass is exposable as a flag, and `python ppo.py --help` lists them with their defaults. The defaults reproduce the phase 4 configuration of Section 5.3.

```
python ppo.py --folder-name runs --reward-type FBEST_IMP_RATIO \
               --dims 5 --num-envs 64 --total-timesteps 100000000
```

The arguments most relevant to reproducing the experiments in Section 5 are:

- `--fids`: BBOB function IDs to train on. Default 1 2 ...24.
- `--iids`: BBOB instance IDs. Default 15 ...114, the training instance set disjoint from the evaluation instances 0–14.
- `--dims`: problem dimensionalities. Default 5.
- `--state-type`: observation space, currently only PSB.
- `--reward-type`: one of `FBEST_IMP`, `FBEST_IMP_RATIO`, `RAW_BEST`, `FBEST_IMP_EVALS` (Section 4.3).
- `--max-lambda`: upper bound on the population size action. Default 512.

- `--num-envs`, `--num-steps`, `--total-timesteps`: parallel environments, steps collected per environment per batch, and total environment steps. Phases 1–3 used `--num-envs 32 --total-timesteps 100000000`; phase 4 used `--num-envs 64 --total-timesteps 1000000000`.
- `--folder-name`: parent directory for checkpoints and TensorBoard logs.
- `--seed`: random seed, default 1.

Weights and Biases tracking is enabled by default. To train without a W&B account, pass `--no-track`; otherwise run `wandb login` first.

Checkpoints are written every 50 batches to `{folder_name}/{run_name}/ppo_{iteration}.pt`, with a final `ppo.pt` at the end of training, where `run_name` encodes the seed, observation space, reward function and timestep budget. The per-batch progress log is appended to `progress.txt` in the same directory. The parent directory must exist before launching, since the script opens the progress file in append mode without creating intermediate directories:

```
mkdir -p runs
```

A.3 Evaluating a Trained Policy

`benchmark.py` evaluates a controller across the full protocol of Section 5 (functions 1–24, instances 0–14, dimensions 2, 3, 5, 10, 20 and 40) using `ioh.Experiment`, and writes IOHProfiler logs to disk. The `run_exp_weights` function evaluates a single checkpoint:

```
python -c "
from benchmark import run_exp_weights
run_exp_weights('runs/<run_name>/ppo_150.pt', 'results', 'DRLPop',
               'DRLPop', 'FBEST_IMP_RATIO phase 1')
"
```

Its parameters are, in order, the path to the `.pt` weights file, the output directory, the folder name created within it, the algorithm name recorded in the logs, and a free-text description. The optional `limit` argument (default 2) is the bound of the raw action range assumed when rescaling the agent’s output into a population size, and must match the `RescaleAction` bound used during training.

`run_exp_folder` evaluates every checkpoint under a directory tree, which is how the phase 4 checkpoint sweep of Table 6 was produced:

```
python -c "
from benchmark import run_exp_folder
run_exp_folder('runs', 'results', 'DRLPop')
"
```

The two baselines are evaluated through the same `ioh.Experiment` path by substituting the controller. `benchmark.py` does not ship a function for this, so the following snippet reproduces the Default and IPOPOP rows of Tables 4 and 5:

```
from ioh import Experiment, ProblemClass, logger
from cmaes import CMAES
from controllers import DefaultPop, IPOPOP
```

```

for controller, name in [(DefaultPop(), 'Default'), (IPOP(), 'IPOP')]:
    Experiment(
        algorithm=CMAES(controller=controller),
        fids=range(1, 25), iids=range(15),
        dims=[2, 3, 5, 10, 20, 40], reps=1,
        problem_class=ProblemClass.BBOB, njobs=-1, logged=True,
        logger_triggers=[logger.trigger.ON_IMPROVEMENT],
        logger_additional_properties=[],
        output_directory='results', folder_name=name,
        algorithm_name=name, algorithm_info='', zip_output=False,
        run_attributes=['success'],
    ).run()

```

Runs are parallelised across cores by `ioh.Experiment (njobs=-1)`. A full 2160-problem evaluation takes several hours; IPOP is the slowest of the three, since it consumes the entire evaluation budget on every problem (Table 5).

A.4 Viewing Results

`results.py` reads the IOHProfiler logs produced above and computes the metrics of Section 5.2. It expects every controller’s log folder to sit directly beneath a directory named `results`, and loads all of them together so that the AOCC tables and ECDF plots compare controllers side by side.

Running the module directly prints the success counts of Table 4 and the mean evaluations per dimension of Table 5:

```
python results.py
```

The remaining functions are called individually and each writes its output to disk:

- `aocc_table_5()`: per-function AOCC at dimension 5 (Table 7), written as LaTeX to `temp.txt`.
- `aocc_table_averaged()`: per-function AOCC averaged over all six dimensions (Table 8), also written to `temp.txt`.
- `saveecdfs()`: the aggregated ECDF of Figure 4 and the per-function curves of Figures 5–7, written to `graphs/ecdfs/`.
- `savefixbudget()`, `savefixtarget()`: fixed-budget and fixed-target curves per function, written to `graphs/fb/` and `graphs/ft/`.

The output directories must exist beforehand:

```
mkdir -p graphs/ecdfs graphs/fb graphs/ft
```

Note that both AOCC functions write to the same `temp.txt`, so the first table should be copied elsewhere before the second is generated.

A.5 Reproducing the Reported Results End to End

```
mkdir -p runs results graphs/ecdfs

# 1. train (phase 1 configuration, ~10M steps)
python ppo.py --folder-name runs --no-track \
              --reward-type FBEST_IMP_RATIO \
              --num-envs 32 --total-timesteps 10000000

# 2. evaluate the trained policy and both baselines into results/
#    (see the snippets above)

# 3. compute metrics
python results.py
```

Training is by far the dominant cost: the phase 1 configuration takes on the order of a day on a single GPU machine, and phase 4 roughly ten times that. Evaluation of a single controller across the full protocol is a matter of hours.

B Supplementary Training and Evaluation Data

This appendix contains the full numerical data underlying the summary discussion in Section 5.3 and Section 5.4: the complete phase 4 checkpoint sweep (Table 6), per function AOCC values at dimension 5 and averaged across all six dimensions (Tables 7 and 8), and per function ECDF curves for all 24 BBOB functions (Figures 5–7).

Reward function, batch	2	3	5	10	20	40	Sum
FBEST_IMP_EVALS 50	65	55	55	59	44	14	292
FBEST_IMP_EVALS 100	22	16	13	15	15	15	96
FBEST_IMP_EVALS 150	23	15	15	15	15	15	98
FBEST_IMP_EVALS 200	17	15	15	15	15	15	92
FBEST_IMP_EVALS 250	17	17	15	15	15	15	94
FBEST_IMP_EVALS 300	22	15	15	15	15	15	97
FBEST_IMP_EVALS 350	17	15	15	15	15	15	92
FBEST_IMP_EVALS 400	16	15	15	15	15	15	91
FBEST_IMP_EVALS 450	21	15	15	15	15	15	96
FBEST_IMP_EVALS 500	17	15	15	15	15	15	92
FBEST_IMP_EVALS 550	19	15	15	15	15	15	94
FBEST_IMP_EVALS 600	21	15	15	15	15	15	96
FBEST_IMP_EVALS 650	23	16	15	15	15	15	99
FBEST_IMP_EVALS 700	17	15	15	15	15	15	92
FBEST_IMP_EVALS 750	16	15	15	15	15	15	91
FBEST_IMP_EVALS 800	20	15	15	15	15	15	95
FBEST_IMP_EVALS 850	19	16	15	15	15	15	95
FBEST_IMP_EVALS 900	18	15	15	15	15	15	93
FBEST_IMP_EVALS 950	18	16	15	15	15	15	94
FBEST_IMP_EVALS 1000	18	15	15	15	15	15	93
FBEST_IMP_EVALS 1050	21	16	15	15	15	15	97
FBEST_IMP_EVALS 1100	17	15	15	15	15	15	92
FBEST_IMP_EVALS 1150	18	15	15	15	15	15	93
FBEST_IMP_EVALS 1200	16	15	15	15	15	15	91
FBEST_IMP_EVALS 1250	25	15	15	15	15	15	100
FBEST_IMP_EVALS 1300	19	15	15	15	15	15	94
FBEST_IMP_EVALS 1350	20	15	15	15	15	15	95
FBEST_IMP_EVALS 1400	19	15	15	15	15	15	94
FBEST_IMP_EVALS 1450	20	15	15	15	15	15	95
FBEST_IMP_EVALS 1500	21	15	15	15	15	15	96
FBEST_IMP_EVALS 1525	19	17	15	15	15	15	96
FBEST_IMP_RATIO 50	66	58	53	68	50	14	309
FBEST_IMP_RATIO 100	198	174	167	155	121	52	867
FBEST_IMP_RATIO 150	207	182	173	164	119	63	908
FBEST_IMP_RATIO 200	207	183	171	152	111	41	865
FBEST_IMP_RATIO 250	211	179	165	144	98	22	819
FBEST_IMP_RATIO 300	203	185	171	136	97	23	815
FBEST_IMP_RATIO 350	191	163	155	141	93	23	766
FBEST_IMP_RATIO 400	172	154	149	133	95	18	721
FBEST_IMP_RATIO 450	174	162	149	141	89	24	739
FBEST_IMP_RATIO 500	179	163	150	141	102	29	764
FBEST_IMP_RATIO 550	186	168	153	138	104	19	768
FBEST_IMP_RATIO 600	196	169	152	133	101	21	772
FBEST_IMP_RATIO 650	190	174	164	144	104	23	799
FBEST_IMP_RATIO 700	192	167	159	146	107	25	796
FBEST_IMP_RATIO 750	187	170	166	142	99	26	790

Table 6: Phase 4 checkpoint sweep: successes by training batch, reward function, and dimension. FBEST_IMP_RATIO training terminated early at batch 750 due to a NaN error.

	DRLPOP	Default	IPOP	Best
Sphere	0.963235	0.995663	0.995678	IPOP
Ellipsoid	0.867302	0.977852	0.977027	Default
Rastrigin	0.479645	0.453050	0.557233	IPOP
BuecheRastrigin	0.451238	0.443172	0.462073	IPOP
LinearSlope	0.997561	0.999139	0.999201	IPOP
AttractiveSector	0.903838	0.988105	0.987253	Default
StepEllipsoid	0.964844	0.575028	0.974506	IPOP
Rosenbrock	0.882546	0.911720	0.978825	IPOP
RosenbrockRotated	0.885303	0.979150	0.978935	Default
EllipsoidRotated	0.864393	0.977896	0.976915	Default
Discus	0.873432	0.976898	0.976833	Default
BentCigar	0.806259	0.951057	0.962826	IPOP
SharpRidge	0.869296	0.937759	0.975485	IPOP
DifferentPowers	0.932375	0.989298	0.989017	Default
RastriginRotated	0.708173	0.452590	0.622250	DRLPOP
Weierstrass	0.831310	0.555465	0.899309	IPOP
Schaffers10	0.914060	0.695376	0.950945	IPOP
Schaffers1000	0.898461	0.589069	0.852672	DRLPOP
GriewankRosenbrock	0.598743	0.515646	0.579619	DRLPOP
Schwefel	0.527120	0.488996	0.533390	IPOP
Gallagher101	0.607636	0.572932	0.788007	IPOP
Gallagher21	0.484736	0.610013	0.702207	IPOP
Katsuura	0.498648	0.499317	0.573104	IPOP
LunacekBiRastrigin	0.447202	0.445566	0.456897	IPOP
Average	0.760723	0.732532	0.822925	IPOP

Table 7: AOCC by function and controller, dimension 5.

	DRLPOP	Default	IPOP	Best
Sphere	0.972241	0.995791	0.995796	IPOP
Ellipsoid	0.822848	0.956899	0.956726	Default
Rastrigin	0.538551	0.452667	0.551553	IPOP
BuecheRastrigin	0.454822	0.428446	0.460998	IPOP
LinearSlope	0.912721	0.999159	0.999251	IPOP
AttractiveSector	0.688637	0.985030	0.984433	Default
StepEllipsoid	0.787876	0.551359	0.827439	IPOP
Rosenbrock	0.847201	0.925860	0.955983	IPOP
RosenbrockRotated	0.845298	0.912805	0.951306	IPOP
EllipsoidRotated	0.819215	0.956987	0.956870	Default
Discus	0.842737	0.953522	0.967195	IPOP
BentCigar	0.827675	0.936205	0.957100	IPOP
SharpRidge	0.765201	0.791690	0.868930	IPOP
DifferentPowers	0.920859	0.983501	0.983255	Default
RastriginRotated	0.613570	0.445566	0.614827	IPOP
Weierstrass	0.759964	0.583997	0.790817	IPOP
Schaffers10	0.878275	0.662583	0.920524	IPOP
Schaffers1000	0.805400	0.563204	0.814870	IPOP
GriewankRosenbrock	0.637344	0.538521	0.615732	DRLPOP
Schwefel	0.575425	0.504519	0.586818	IPOP
Gallagher101	0.605904	0.523970	0.758302	IPOP
Gallagher21	0.545896	0.525609	0.637524	IPOP
Katsuura	0.532479	0.543594	0.617658	IPOP
LunacekBiRastrigin	0.444947	0.429666	0.448613	IPOP
Average	0.726879	0.714631	0.800938	IPOP

Table 8: AOCC by function and controller, averaged over all six dimensions.

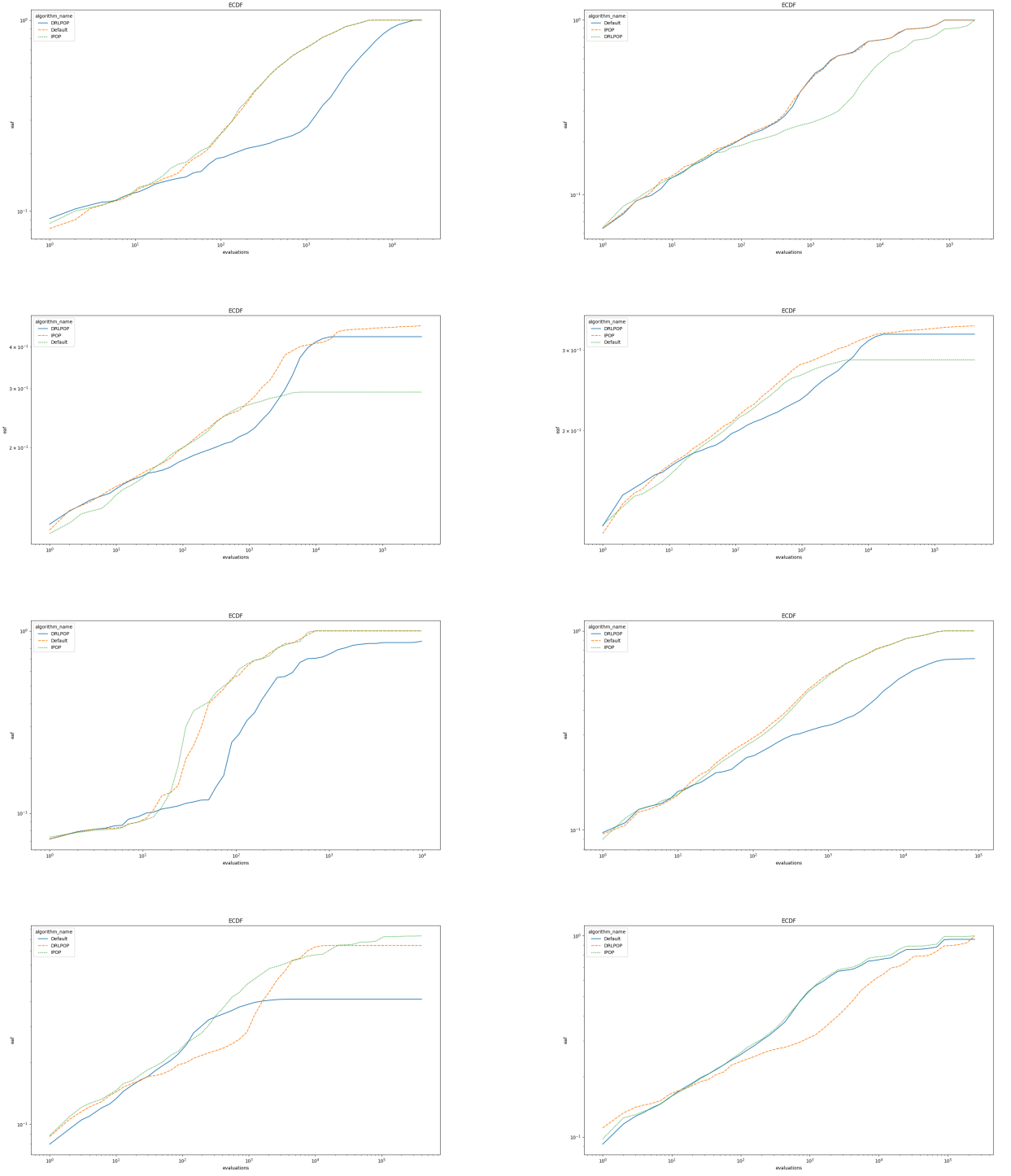


Figure 5: ECDF results for BBOB functions 1–8.

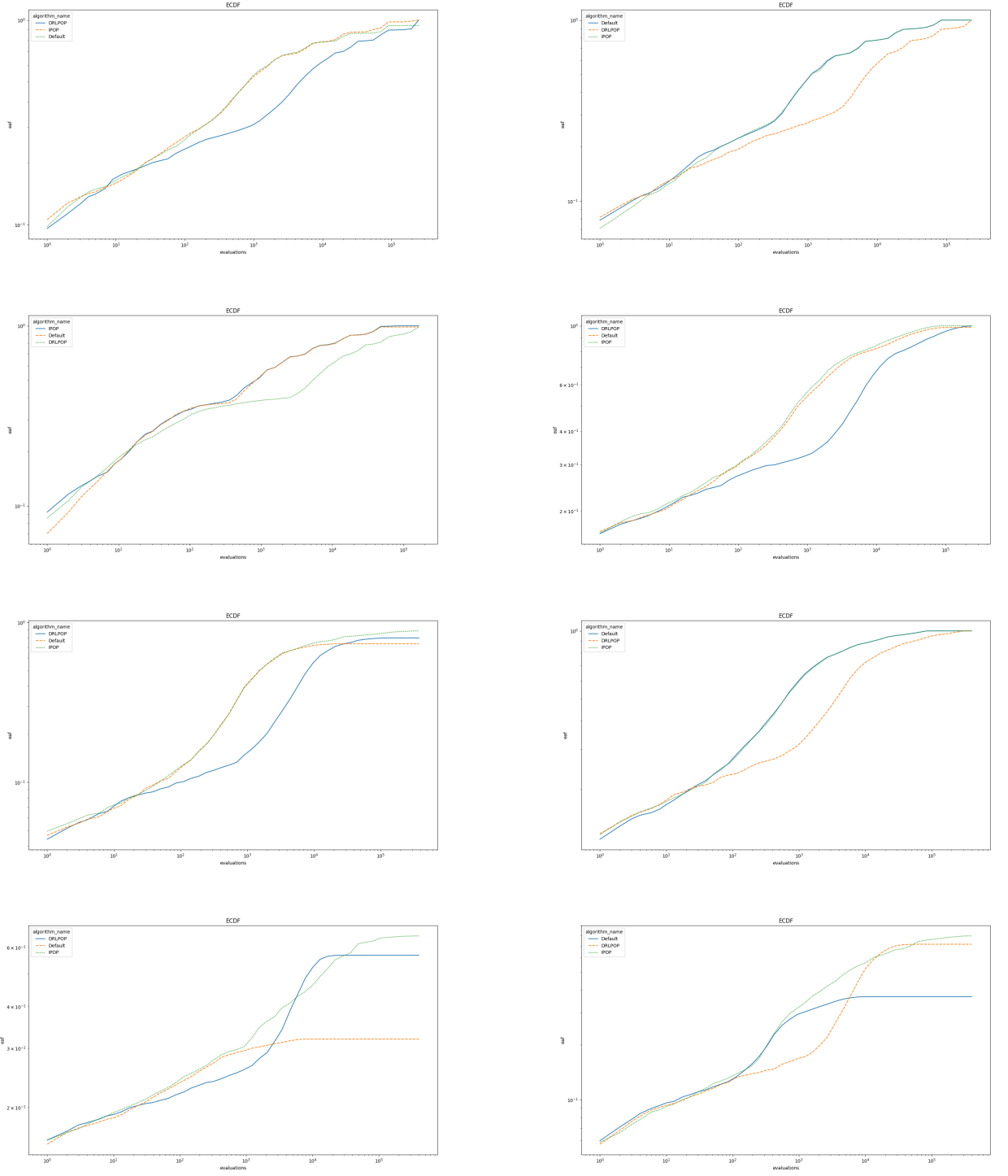


Figure 6: ECDF results for BBOB functions 9–16.

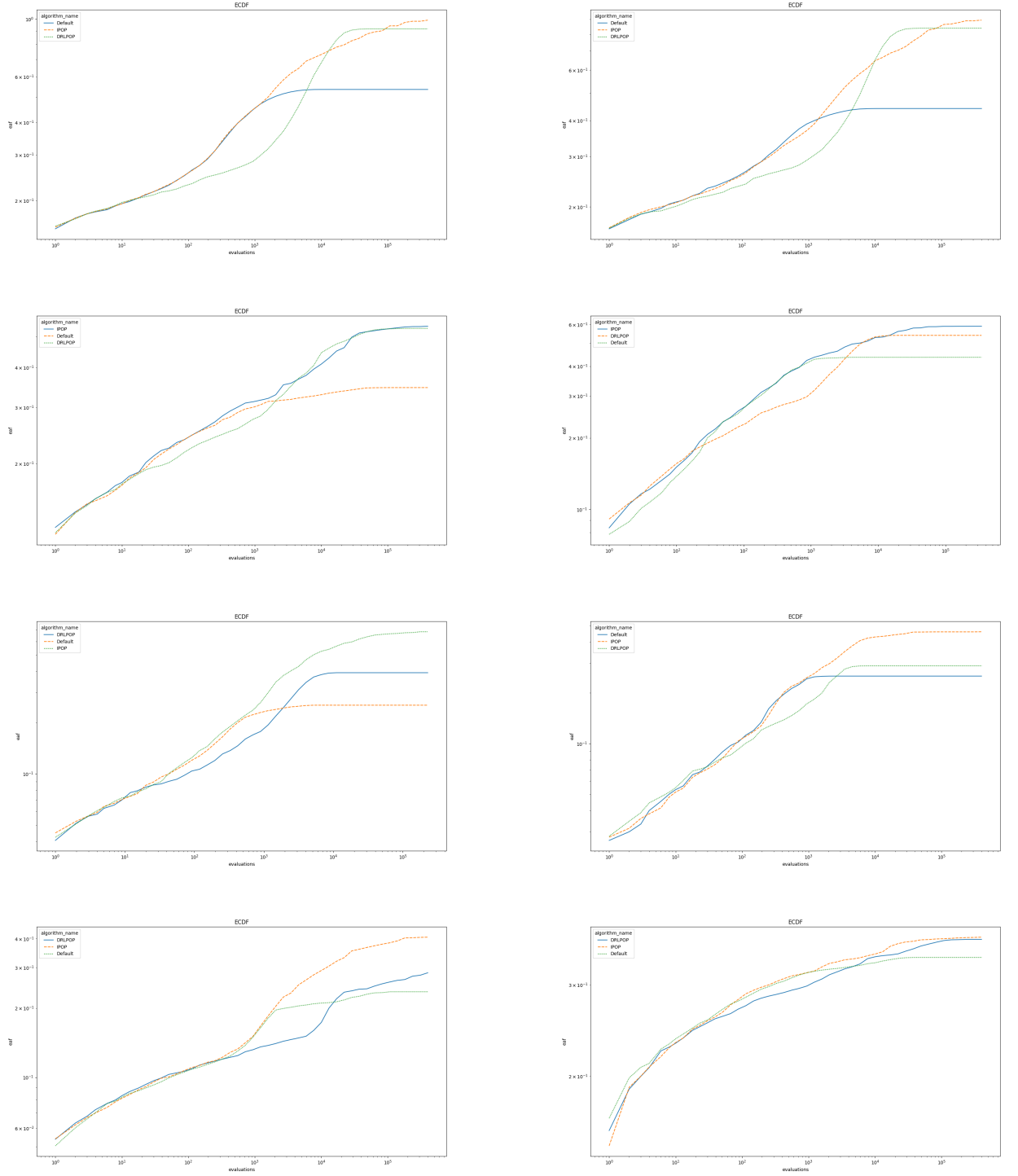


Figure 7: ECDF results for BBOB functions 17–24.